THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms.**SUBATHRA D (20524UCSC045) ,** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

Course In-Charge Head of the Department

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Annexures :**		
	• **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX NO: 01 CREATION OF DDL OPERATIONS USING LIBRARY TABLE

DATE:02.09.2026

Aim

To create a Library table, insert book records, modify the table structure and perform DDL operations such as CREATE, ALTER, DROP and TRUNCATE.

Software Used

Oracle / MySQL / SQLite

Description

DDL stands for **Data Definition Language**. It is used to create, modify and remove the structure of database objects such as tables.

DDL COMMANDS

- CREATE
- ALTER
- DROP
- TRUNCATE

CREATE

CREATE command is used to create a new table and define its columns and constraints.

Syntax

```
CREATE TABLE table_name ( column1
    datatype constraint, column2 datatype
    constraint, column3 datatype);
```

Table Structure

```
CREATE TABLE Library (
    Book_ID INTEGER PRIMARY KEY,
    Book_Name VARCHAR(50),
```

Author VARCHAR(40), Price

INTEGER);

Query

```
CREATE TABLE Library (
    Book_ID INTEGER PRIMARY KEY,
    Book_Name VARCHAR(50),
    Author VARCHAR(40), Price
    INTEGER);
```

Table Created Sample

Data

```
INSERT INTO Library VALUES (101,'Database Systems','Ramesh',450);
INSERT INTO Library VALUES (102,'Java Programming','Kumar',520);
INSERT INTO Library VALUES (103,'Web Technology','Priya',400
INSERT INTO Library VALUES (104,'Python Basics','Arun',480);
INSERT INTO Library VALUES (105,'Computer Networks','Divya',550);
```

SELECT

```
SELECT * FROM Library;
```

Output

BOOK_ID	BOOK_NAME	AUTHOR	PRICE
101	Database Systems	Ramesh	450
102	Java Programming	Kumar	520
103	Web Technology	Priya	400
104	Python Basics	Arun	480
105	Computer Networks	Divya	550

ALTER

ALTER command is used to change the structure of an existing table. It can be used to add, modify or remove columns.

Syntax

ALTER TABLE table_name ADD column_name datatype;

Query

ALTER TABLE Library ADD Publisher VARCHAR(40);

Output

Table altered successfully.

TRUNCATE

TRUNCATE command is used to remove all records from a table while retaining its structure.

Syntax

TRUNCATE TABLE table_name;

Query

TRUNCATE TABLE Library;

Output

Table truncated successfully.

DROP

DROP command is used to permanently remove a table from the database.

Syntax

DROP TABLE table_name;

Query

DROP TABLE Library;

Output

Table dropped successfully.

Result

Thus, the DDL operations **CREATE, ALTER, TRUNCATE and DROP** were successfully performed on the Library table.

EX. NO: 02 DML COMMANDS USING STUDENT TABLE

DATE: 05.09.2026

Aim

To create a Student table and perform DML operations such as INSERT, UPDATE and DELETE.

Software Used

Oracle / MySQL / SQLite

Description

DML stands for **Data Manipulation Language**. It is used to insert, modify and delete data stored in database tables.

DML COMMANDS

- INSERT
- UPDATE
- DELETE

Table Structure

CREATE TABLE Student (Student_ID

INTEGER, Student_Name

VARCHAR(40),

Department VARCHAR(30),Marks INTEGER);

Table Created INSERT

INSERT command is used to add new records to a table.

Syntax

INSERT INTO table_name

VALUES (value1,value2,value3,value4);

Query

INSERT INTO Student VALUES (1,'Anitha','BCA',85);

INSERT INTO Student VALUES (2,'Bharath','BCA',78);

INSERT INTO Student VALUES (3,'Charan','BSc CS',92);

INSERT INTO Student VALUES (4,'Deepa','BSc CS',88);

INSERT INTO Student VALUES (5,'Elango','BCA',74);

Display Records SELECT *

FROM Student; **Output**

ID	**NAME**	**DEPARTMENT**	**MARKS**
1	Anitha	BCA	85
2	Bharath	BCA	78
3	Charan	BSc CS	92
4	Deepa	BSc CS	88
5	Elango	BCA	74

UPDATE

UPDATE command is used to modify existing records.

Syntax

UPDATE table_name

SET column_name=value

WHERE condition;

Query

UPDATE Student SET

Marks=90

WHERE Student_ID=2;

Output

1 row updated.

DELETE

DELETE command is used to remove a particular record.

Syntax

DELETE FROM table_name WHERE

condition;

Query

DELETE FROM Student

WHERE Student_ID=5;

Display Final Records

SELECT * FROM Student;

Output

I D	NAME	DEPARTMENT	MARKS
1	Anitha	BCA	85
2	Bharath	BCA	90
3	Charan	BSc CS	92
4	Deepa	BSc CS	88

Result

Thus, the DML operations **INSERT, UPDATE and DELETE** were successfully performed on the Student table.

EX. NO: 03 DQL COMMANDS USING EMPLOYEE TABLE

DATE: 08.09.2026

Aim

To retrieve and manipulate data using SELECT, WHERE, ORDER BY, GROUP BY and HAVING clauses.

Software Used

Oracle / MySQL / SQLite

Description

DQL stands for **Data Query Language**. It is mainly used to retrieve data from database tables.

DQL COMMANDS

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

Table Structure

```
CREATE TABLE Employee_Details ( Emp_ID
    INTEGER,
Emp_NameVARCHAR(40),
Department    VARCHAR(30),    Salary
INTEGER);
```

Sample Data

INSERT INTO Employee_Details VALUES (201,'Vijay','IT',35000);

INSERT INTO Employee_Details VALUES (202,'Meena','HR',30000);

INSERT INTO Employee_Details VALUES (203,'Rahul','IT',45000);

INSERT INTO Employee_Details VALUES (204,'Swetha','Sales',28000);

INSERT INTO Employee_Details VALUES (205,'Naveen','HR',36000);

SELECT

SELECT is used to retrieve records from a table.

Syntax

SELECT column_name FROM

table_name;

Query

SELECT * FROM Employee_Details;

WHERE

WHERE is used to retrieve records that satisfy a condition.

SELECT *FROM Employee_DetailsWHERE Salary > 30000;

ORDER BY

ORDER BY is used to arrange records in ascending or descending order. SELECT *

FROM Employee_Details

ORDER BY Salary DESC;

GROUP BY

GROUP BY is used to group records having the same values. SELECT

Department, AVG(Salary)

FROM Employee_Details

GROUP BY Department;

HAVING

HAVING is used to apply a condition to grouped records. SELECT

Department, AVG(Salary)

FROM Employee_Details GROUP BY

Department HAVING AVG(Salary) >

32000;

OUTPUT

DEPARTMENT AVERAGE SALARY

---------- ----------

IT 40000

HR

Result

Thus, the DQL clauses **SELECT, WHERE, ORDER BY, GROUP BY and HAVING** were successfully execute

EX. NO: 04 TRANSACTION CONTROL COMMANDS

DATE:11.09.2026

Aim

To perform transaction control operations using COMMIT, ROLLBACK and SAVEPOINT.

Software Used

Oracle / MySQL

Description

Transaction Control Language is used to manage transactions in a database.

TRANSACTION COMMANDS

- COMMIT
- ROLLBACK
- SAVEPOINT

Table Structure

```
CREATE TABLE Bank_Account (
    Account_No INTEGER,
    Holder_Name VARCHAR(40),
    Balance INTEGER);
```

Sample Data

```
INSERT INTO Bank_Account VALUES (301,'Karthik',25000);

INSERT INTO Bank_Account VALUES (302,'Harini',32000);

INSERT INTO Bank_Account VALUES (303,'Sanjay',18000);

INSERT INTO Bank_Account VALUES (304,'Keerthi',45000);
```

INSERT INTO Bank_Account VALUES (305,'Manoj',27000);

COMMIT

COMMIT permanently saves the transaction. COMMIT;

SAVEPOINT

SAVEPOINT creates a temporary point within a transaction. SAVEPOINT S1;

UPDATE

UPDATE Bank_Account SET

Balance=Balance-5000 WHERE

Account_No=301;

Output

Account 301 balance = 20000

ROLLBACK

ROLLBACK is used to undo changes made after a savepoint. ROLLBACK TO S1;

Output

Account 301 balance = 25000

Result

Thus, the transaction control commands **COMMIT, ROLLBACK and SAVEPOINT** were successfully implemented.

EX. NO: 05 IMPLEMENTATION OF JOINS

DATE:13.09.2026

Aim

To retrieve related data from two or more tables using different types of joins.

Software Used

Oracle / MySQL / SQLite

Description

A JOIN is used to combine records from two or more tables based on a related column.

TYPES OF JOINS

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- SELF JOIN

Department Table

```
CREATE TABLE Department_Info (
    Dept_ID INTEGER, Dept_Name VARCHAR(30));
```

Sample Data

```
INSERT INTO Department_Info VALUES (1,'Computer');

INSERT INTO Department_Info VALUES (2,'Commerce');

INSERT INTO Department_Info VALUES (3,'Science');
```

INSERT INTO Department_Info VALUES (4,'Management');

INSERT INTO Department_Info VALUES (5,'English');

Staff Table

CREATE TABLE Staff_Info (

Staff_ID INTEGER, Staff_Name

VARCHAR(40), Dept_ID

INTEGER,

Salary INTEGER);

Sample Data

INSERT INTO Staff_Info VALUES (401,'Ravi',1,42000);

INSERT INTO Staff_Info VALUES (402,'Latha',2,38000);

INSERT INTO Staff_Info VALUES (403,'Mohan',1,45000);

INSERT INTO Staff_Info VALUES (404,'Rekha',3,40000);

INSERT INTO Staff_Info VALUES (405,'Suresh',4,36000);

INNER JOIN

INNER JOIN returns matching records from both tables.

SELECT S.Staff_Name,D.Dept_Name FROM

Staff_Info S

INNER JOIN Department_Info D ON

S.Dept_ID=D.Dept_ID;

Output

STAFF NAME	**DEPARTMENT**

Ravi	Computer
Latha	Commerce
Mohan	Computer
Rekha	Science
Suresh	Management

LEFT JOIN

SELECT S.Staff_Name,D.Dept_Name FROM

Staff_Info S

LEFT JOIN Department_Info D ON

S.Dept_ID=D.Dept_ID;

RIGHT JOIN

SELECT S.Staff_Name,D.Dept_Name FROM

Staff_Info S

RIGHT JOIN Department_Info D
ON S.Dept_ID=D.Dept_ID;

FULL OUTER JOIN

SELECT S.Staff_Name,D.Dept_Name FROM

Staff_Info S

FULL OUTER JOIN Department_Info D ON

S.Dept_ID=D.Dept_ID;

SELF JOIN

SELF JOIN joins a table with itself

. CREATE TABLE Staff_Manager (

Staff_ID INTEGER, Staff_Name

```
    VARCHAR(40), Manager_ID
    INTEGER);

INSERT INTO Staff_Manager VALUES (501,'Ajay',NULL);
INSERT INTO Staff_Manager VALUES (502,'Bala',501);
INSERT INTO Staff_Manager VALUES (503,'Cathy',501);
INSERT INTO Staff_Manager VALUES (504,'Dinesh',502);
INSERT INTO Staff_Manager VALUES (505,'Esha',502);
SELECT E.Staff_Name AS Employee,
      M.Staff_Name AS Manager FROM
Staff_Manager E
LEFT JOIN Staff_Manager M ON
E.Manager_ID=M.Staff_ID;
```

Output

EMPLOYEE	**MANAGER**
Ajay	NULL
Bala	Ajay
Cathy	Ajay
Dinesh	Bala
Esha	Bala

Result

Thus, the different types of joins were successfully implemented.

EX. NO: 06 IMPLEMENTATION OF SQL FUNCTIONS

DATE:15.09.2026

Aim

To implement aggregate, string and date functions using SQL.

Software Used

Oracle / MySQL / SQLite

Description

SQL functions are predefined operations used to perform calculations, manipulate strings and process dates.

TYPES OF FUNCTIONS

- Aggregate Functions
- String Functions
- Date Functions

Table Structure

```
CREATE TABLE Product ( Product_ID
    INTEGER, Product_Name
    VARCHAR(40), Category
    VARCHAR(30),
Price INTEGER, Purchase_Date DATE);
```

Sample Data

```
INSERT INTO Product VALUES(601,'Keyboard','Computer',800,'2026-01-10');

INSERT INTO Product VALUES (602,'Mouse','Computer',500,'2026-02-15');

INSERT INTO Product VALUES (603,'Monitor','Computer',7500,'2026-03-20');

INSERT INTO Product VALUES (604,'Printer','Office',6000,'2026-04-12');
```

INSERT INTO Product VALUES (605,'Speaker','Audio',2200,'2026-05-18');

Aggregate Functions

COUNT, SUM, AVG, MAX and MIN are aggregate functions.

```
SELECT COUNT(*) AS Total_Products,
    SUM(Price) AS Total_Price,
     AVG(Price) AS Average_Price, MAX(Price)
    AS Maximum_Price, MIN(Price)
     AS Minimum_PriceFROM Product;
```

Output

TOTAL PRODUCTS : 5

TOTAL PRICE: 17000

AVERAGE PRICE : 3400

MAXIMUM PRICE : 7500

MINIMUM PRICE : 500

String Functions

```
SELECT Product_Name, UPPER(Product_Name),
    LOWER(Product_Name), LENGTH(Product_Name)
FROM Product;
```

Date Functions

```
SELECT Product_Name,
    Purchase_Date
FROM Product;
```

Result

Thus, the aggregate, string and date functions were successfully implemented.

EX. NO: 07 SUBQUERIES AND NESTED QUERIES

DATE:17.09.2026

Aim

To retrieve data using subqueries and nested queries.

Software Used

Oracle / MySQL / SQLite

Description

A subquery is a query written inside another SQL query. The inner query provides a result to the outer query.

Table Structure

```
CREATE TABLE Course (
    Course_ID  INTEGER,  Course_Name
    VARCHAR(40), Duration INTEGER,
    Fee INTEGER);
```

Sample Data

```
INSERT INTO Course VALUES (701,'Python',6,12000);
INSERT INTO Course VALUES (702,'Java',6,15000);
INSERT INTO Course VALUES (703,'Web Development',8,18000);
INSERT INTO Course VALUES (704,'Cyber Security',10,25000);
INSERT INTO Course VALUES (705,'Data Analytics',8,20000);
```

Syntax

```
SELECT column_name FROM
```

table_name WHERE

column_name >(SELECT

aggregate_function(column_nam

e) FROM table_name);

Query

```
SELECT Course_Name,Fee
FROM Course
WHERE Fee >
      (SELECTAVG(Fee)
       FROM Course);
```

Output

COURSE NAME	**FEE**
Web Development	18000
Cyber Security	25000
Data Analytics	20000

Result

Thus, the subquery was successfully executed to retrieve courses having fees greater than the average fee.

EX. NO: 08 VIEWS, INDEXES AND SEQUENCES

DATE: 20.09.2026

Aim

To create and implement Views, Indexes and Sequences in a database.

Software Used

Oracle / MySQL

Description

A **View** is a virtual table based on a SQL query. An **Index** improves data retrieval speed. A **Sequence** generates unique numeric values.

Table Structure

```
CREATE TABLE Orders (
    Order_ID INTEGER, Customer_Name
    VARCHAR(40), Product_Name
    VARCHAR(40), Amount INTEGER);
```

Sample Data

```
INSERT INTO Orders VALUES (801,'Arun','Laptop',55000);

INSERT INTO Orders VALUES (802,'Priya','Tablet',25000);

INSERT INTO Orders VALUES (803,'Kavin','Mobile',18000);

INSERT INTO Orders VALUES (804,'Divya','Printer',12000);
 INSERT INTO Orders VALUES (805,'Rahul','Monitor',15000);
```

VIEW

Syntax

CREATE VIEW view_name AS SELECT

column_name

FROM table_name

WHERE condition;

Query

CREATE VIEW Expensive_Orders AS

SELECT Order_ID,Customer_Name,Product_Name,Amount FROM Orders

WHERE Amount>20000;

SELECT * FROM Expensive_Orders;

Output

ORDER ID	CUSTOMER	PRODUCT	AMOUNT
801	Arun	Laptop	55000
802	Priya	Tablet	25000

INDEX

CREATE INDEX Order_Customer_Index ON

Orders(Customer_Name);

Output

Index created successfully.

SEQUENCE

CREATE SEQUENCE Order_Sequence START

WITH 806

INCREMENT BY 1;

```
SELECT Order_Sequence.NEXTVAL
FROM DUAL;
```

Output

806

Result

Thus, Views, Indexes and Sequences were successfully created and implemented.

EX. NO: 09 STORED PROCEDURES AND FUNCTIONS

DATE: 21.09.2026

Aim

To create and execute stored procedures and user-defined functions using PL/SQL.

Software Used

Oracle

Description

A stored procedure is a named PL/SQL program stored in the database. A function is a stored program that returns a value.

Table Structure

```
CREATE TABLE Salary_Info (
    Emp_ID INTEGER, Emp_Name
    VARCHAR(40), Monthly_Salary
    INTEGER);
```

Sample Data

```
INSERT INTO Salary_Info VALUES (901,'Asha',30000);

INSERT INTO Salary_Info VALUES (902,'Binu',35000);

INSERT INTO Salary_Info VALUES (903,'Chitra',40000);

INSERT INTO Salary_Info VALUES (904,'Dharma',32000);

INSERT INTO Salary_Info VALUES (905,'Fathima',38000);
```

STORED PROCEDURE

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name IS
BEGIN
    statements;
END;
```

/

Query

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE Show_Employees IS
BEGIN
    FOR R IN (SELECT * FROM Salary_Info) LOOP
        DBMS_OUTPUT.PUT_LINE(
            R.Emp_ID || ' ' ||
  R.Emp_Name || ' ' ||

R.Monthly_Salary);

END LOOP;
END;
```

Execute

```
EXEC Show_Employees;
```

Output

901 Asha 30000

902 Binu 35000

903 Chitra 40000

904 Dharma 32000

905 Fathima 38000

FUNCTION

```
CREATE OR REPLACE FUNCTION Annual_Salary
```

```
(P_Salary NUMBER)
RETURN NUMBER IS
BEGIN
    RETURN P_Salary*12;
    END;
    /
```

Query

```
SELECT Emp_Name, Annual_Salary(Monthly_Salary)
FROM Salary_Info;
```

Output

EMP NAME	**ANNUAL SALARY**
Asha	360000
Binu	420000
Chitra	480000
Dharma	384000
Fathima	456000

Result

Thus, the stored procedure and user-defined function were successfully created and executed.

EX. NO: 10 IMPLEMENTATION OF TRIGGERS

DATE: 24.09.2026

Aim

To create a trigger that automatically records changes made to a database table.

Software Used

Oracle

Description

A trigger is a PL/SQL program that automatically executes when an INSERT, UPDATE or DELETE event occurs on a table.

Table Structure

```
CREATE TABLE Inventory ( Item_ID
    INTEGER,
    Item_Name VARCHAR(40),
    Quantity INTEGER,
    Price INTEGER);
```

Audit Table

```
CREATE TABLE Inventory_Audit ( Item_ID
    INTEGER,
Action_Name VARCHAR(20),
Action_Date DATE);
```

Trigger Syntax

```
CREATE OR REPLACE TRIGGER trigger_name AFTER
INSERT ON table_name
FOR EACH ROW
BEGIN
    statements;
END;
```

/

Query

```
CREATE OR REPLACE TRIGGER Inventory_Insert_Trigger AFTER
INSERT ON Inventory
FOR EACH ROW
BEGIN
    INSERT INTO Inventory_Audit
    VALUES (:NEW.Item_ID,'INSERT',SYSDATE);  END;
/
```

Sample Data

```
INSERT INTO Inventory VALUES (1001,'Pen Drive',20,600);

INSERT INTO Inventory VALUES (1002,'Hard Disk',10,4500);

INSERT INTO Inventory VALUES (1003,'Keyboard',15,800);

INSERT INTO Inventory VALUES (1004,'Mouse',25,500);

INSERT INTO Inventory VALUES (1005,'Webcam',8,2200);
```

Display Audit Records SELECT *

FROM Inventory_Audit; **Output**

ITEM ID	**ACTION**	**DATE**
1001	INSERT	Current Date
1002	INSERT	Current Date
1003	INSERT	Current Date
1004	INSERT	Current Date

1005	INSERT	Current Date

Result

Thus, the trigger was successfully created and automatically recorded the INSERT operations.

EX. NO: 11 CURSOR IMPLEMENTATION

DATE:26.09.2026

Aim

To implement an explicit cursor for retrieving and processing multiple records.

Software Used

Oracle

Description

A cursor is used to process the rows returned by a query one row at a time.

Table Structure

```
CREATE TABLE Library_Member ( Member_ID
    INTEGER, Member_Name VARCHAR(40),
    Books_Issued INTEGER);
```

Sample Data

```
INSERT INTO Library_Member VALUES (1101,'Gokul',3);

INSERT INTO Library_Member VALUES (1102,'Hema',5);

INSERT INTO Library_Member VALUES (1103,'Irfan',2);

INSERT INTO Library_Member VALUES (1104,'Jaya',4);
INSERT INTO Library_Member VALUES (1105,'Lokesh',1);
```

Cursor Syntax

```
DECLARE
    CURSOR cursor_name IS SELECT
        statement;
BEGIN
```

```
    OPEN cursor_name;
    FETCH cursor_name INTO variables; CLOSE
    cursor_name;
END;
/
```

Program

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR Member_Cursor IS
        SELECT Member_ID,Member_Name,Books_Issued FROM
        Library_Member;
    V_ID Library_Member.Member_ID%TYPE; V_NAME
    Library_Member.Member_Name%TYPE; V_BOOKS
    Library_Member.Books_Issued%TYPE;

BEGIN
    OPEN Member_Cursor;

    LOOP
        FETCH Member_Cursor
        INTO V_ID,V_NAME,V_BOOKS;

        EXIT WHEN Member_Cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            V_ID || ' ' || V_NAME || '
            ' || V_BOOKS );
    END LOOP;
```

```
    CLOSE Member_Cursor; END;
/
```

Output

1101 Gokul 3

1102 Hema 5

1103 Irfan 2

1104 Jaya 4

1105 Lokesh 1

PL/SQL procedure successfully completed.

Result

Thus, the explicit cursor was successfully implemented to retrieve records one by one.

EX. NO: 12 PL/SQL EXCEPTION HANDLING

DATE:26.09.2026

Aim

To implement exception handling in PL/SQL using predefined exceptions.

Software Used

Oracle

Description

Exception handling is used to handle errors that occur during the execution of a PL/SQL program.

Types of Exceptions

- ZERO_DIVIDE
- NO_DATA_FOUND
- TOO_MANY_ROWS
- OTHERS

ZERO_DIVIDE

Program

```
SET SERVEROUTPUT ON;

DECLARE

    A NUMBER := 100;

    B NUMBER := 0;

    C NUMBER;


BEGIN

    C := A/B;
```

```
    DBMS_OUTPUT.PUT_LINE(
        'Result = ' || C );

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE(
            'ERROR: Division by zero is not allowed.');

END;
/
```

Output

ERROR: Division by zero is not allowed. PL/SQL procedure successfully completed.

NO_DATA_FOUND

```
SET SERVEROUTPUT ON;

DECLARE
    V_NAME VARCHAR2(40);

BEGIN
    SELECT Student_Name
    INTO V_NAME
    FROM Student
    WHERE Student_ID=999;

    DBMS_OUTPUT.PUT_LINE(V_NAME);

EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE(
        'ERROR: No student record found.');
END;
/
```

Output

ERROR: No student record found.

Result

Thus, PL/SQL exception handling was successfully implemented using predefined exceptions.

EX. NO: 13 DATABASE NORMALIZATION – 1NF, 2NF, 3NF AND BCNF

DATE:28.09.2026

Aim

To normalize a database into **1NF, 2NF, 3NF and BCNF** to reduce redundancy and improve data integrity.

Software Used

Oracle/MySQL

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion and update anomalies.

UNNORMALIZED TABLE

STUDENT ID	STUDENT NAME	COURSE	FACULTY
1	Arun	DBMS, Java	Kumar, Priya
2	Bala	Python	Meena
3	Chitra	DBMS	Kumar
4	Divya	Java, Python	Priya, Meena
5	Ezhil	DBMS	Kumar

The above table contains multiple values in a single field.

FIRST NORMAL FORM – 1NF

In 1NF, each column must contain atomic values.

STUDENT ID	STUDENT NAME	COURSE	FACULTY

1	Arun	DBMS	Kumar
1	Arun	Java	Priya
2	Bala	Python	Meena
3	Chitra	DBMS	Kumar
4	Divya	Java	Priya
4	Divya	Python	Meena
5	Ezhil	DBMS	Kumar

SECOND NORMAL FORM – 2NF

2NF removes partial dependency from a relation.

Student Table

STUDENT ID	STUDENT NAME
1	Arun
2	Bala
3	Chitra
4	Divya
5	Ezhil

COURSE ID	COURSE NAME
1	DBMS
2	Java
3	Python

Enrollment Table

STUDENT ID	COURSE ID
1	1
1	2
2	3
3	1
4	2

THIRD NORMAL FORM – 3NF

3NF removes transitive dependencies.

Faculty Table

FACULTY ID	FACULTY NAME
1	Kumar
2	Priya
3	Meena

Course Table

COURSE ID	COURSE NAME	FACULTY ID
1	DBMS	1
2	Java	2
3	Python	3

BOYCE-CODD NORMAL FORM – BCNF

A relation is in BCNF if every determinant is a candidate key. The normalized

database contains:

STUDENT

|

| Student_ID

|

ENROLLMENT

|

| Course_ID

|

COURSE

|

|

Faculty_ID

|

FACULTY

Y

COURSE

|

| Faculty_ID

|

FACULTY

Result

Thus, the database was successfully normalized into **1NF, 2NF, 3NF and BCNF**, reducing redundancy and improving database integrity.

KHAN ACADEMY

1. PLAYLIST MAKER

PROGRAM

```
CREATE TABLE artists ( id INTEGER PRIMARY

   KEY AUTOINCREMENT, name TEXT, country

   TEXT, genre TEXT);


     INSERT INTO artists (name, country, genre)
 VALUES ("Taylor Swift", "US", "Pop"); INSERT
         INTO artists (name, country, genre)
   VALUES ("Led Zeppelin", "US", "Hard rock"); INSERT
INTO artists (name, country, genre)
   VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
   VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre) VALUES
   ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre) VALUES
   ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre) VALUES
   ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre) VALUES
   ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)

   VALUES ("Rihanna", "US", "Pop");


INSERT INTO artists (name, country, genre) VALUES
("Guns N' Roses", "US", "Hard rock");


INSERT INTO artists (name, country, genre)
VALUES ("Gloria Estefan", "US", "Pop");

 INSERT INTO artists (name, country, genre)
VALUES ("Bob Marley", "Jamaica", "Reggae");
```

```
CREATE TABLE songs ( id INTEGER PRIMARY
   KEY AUTOINCREMENT,
   artist TEXT, title
   TEXT);


INSERT INTO songs (artist, title)
   VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
   VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
   VALUES ("Celine Dion", "My heart will go on"); INSERT
INTO songs (artist, title)
   VALUES ("Celine Dion", "A new day has come"); INSERT
INTO songs (artist, title)
   VALUES ("Shania Twain", "Party for two"); INSERT
INTO songs (artist, title)
   VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");

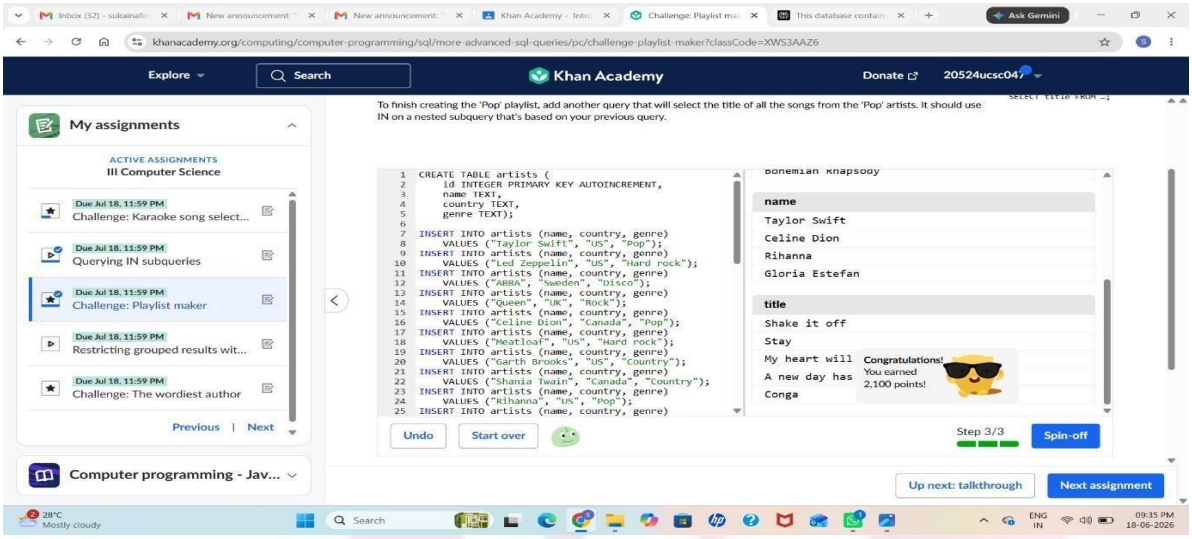

INSERT INTO songs (artist, title)
   VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title) VALUES
   ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
   VALUES ("Queen", "Bohemian Rhapsody"); INSERT
INTO songs (artist, title)
   VALUES ("Guns N' Roses", "Don't cry");


SELECT title FROM songs WHERE
artist = "Queen";


SELECT name FROM artists WHERE genre = 'Pop';


SELECT title FROM songs WHERE artist IN (SELECT name FROM artists WHERE genre
= 'Pop');
```

OUTPUT


Explore
Search
Khan Academy
Donate
20524ucsc047
My assignments
ACTIVE ASSIGNMENTS
III Computer Science
Due Jul 18, 11:59 PM
Challenge: Karaoke song select...
Due Jul 18, 11:59 PM
Querying IN subqueries
Due Jul 18, 11:59 PM
Challenge: Playlist maker
Due Jul 18, 11:59 PM
Restricting grouped results wit...
Due Jul 18, 11:59 PM
Challenge: The wordiest author
Previous | Next
Computer programming - Jav...
To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.
name
Taylor Swift
Celine Dion
Rihanna
Gloria Estefan
title
Shake it off
Stay
My heart will
A new day has
Conga
Congratulations!
You earned
2,100 points!
Undo
Start over
Step 3/3
Spin-off
Up next: talkthrough
Next assignment

2. RESTRICTING GROUPING RESULTS WITH HAVING

PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,

   type TEXT, minutes

   INTEGER, calories

   INTEGER,

   heart_rate INTEGER);


INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 115,
110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 45,
105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30,
70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25,
72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);


SELECT * FROM exercise_logs;


SELECT type, SUM(calories) AS total_calories FROM exercise_logs GROUP BY type;

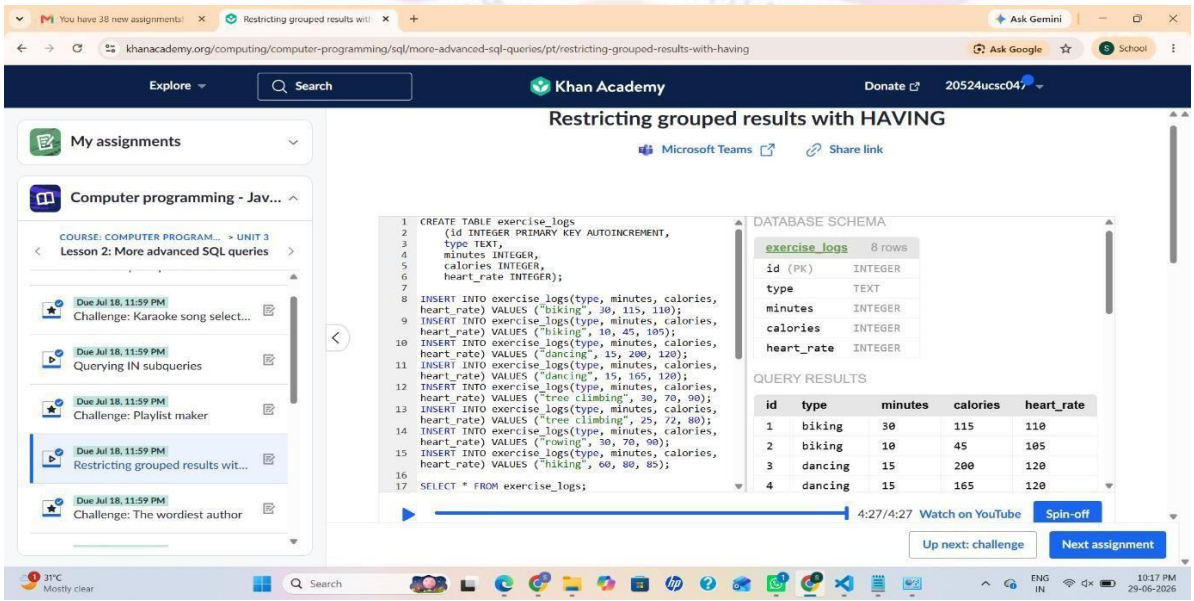

SELECT type, SUM(calories) AS total_calories FROM exercise_logs
```

```
    GROUP BY type
    HAVING total_calories > 150
    ;
```

```
SELECT type, AVG(calories) AS avg_calories FROM exercise_logs
    GROUP BY type
    HAVING avg_calories > 70
    ;
```

```
SELECT type FROM exercise_logs GROUP BY type HAVING COUNT(*) >= 2;
```

OUTPUT



3. THE WORDIEST AUTHOR

PROGRAM

```
CREATE TABLE books ( id INTEGER PRIMARY

   KEY AUTOINCREMENT,

   author TEXT, title
   TEXT,
   words INTEGER);



INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);

INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);

INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637); INSERT
INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045); INSERT
INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);



INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501); INSERT
INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);


   INSERT INTO books (author, title, words)

   VALUES ("Stephenie Meyer", "Eclipse", 147930); INSERT
INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);



INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "The Hobbit", 95022); INSERT
INTO books (author, title, words)
```

VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227); INSERT INTO books (author, title, words)
VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

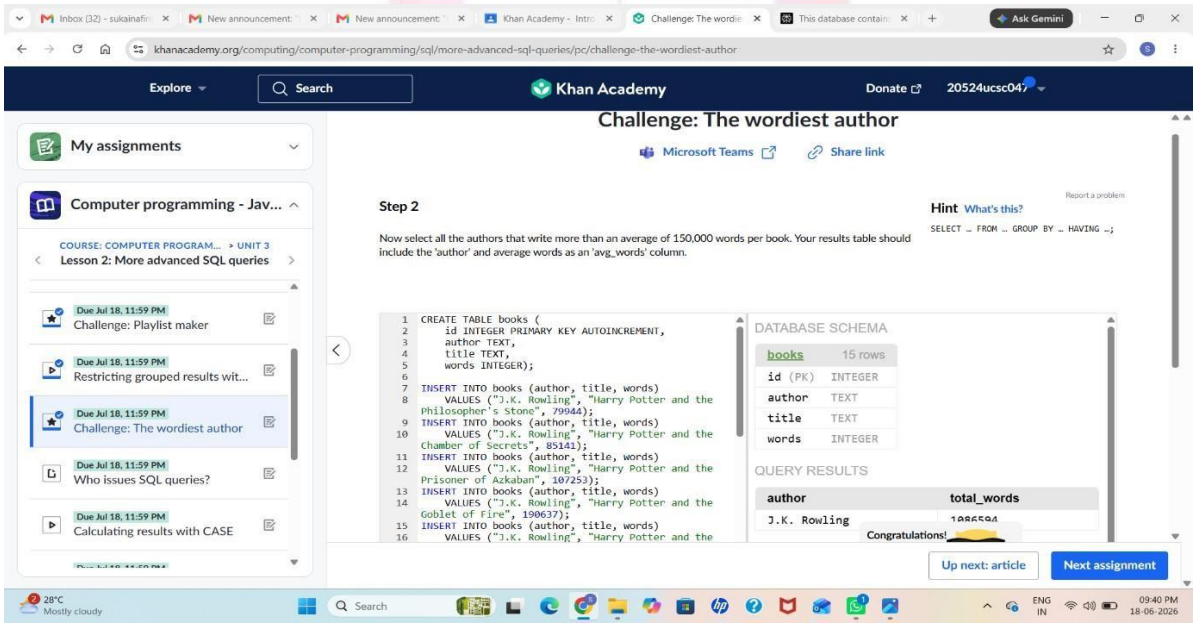
SELECT author, SUM(words) AS total_words FROM

books

GROUP BY author
HAVING total_words > 1000000;

SELECT author, AVG(words) AS avg_words FROM books
GROUP BY author
HAVING avg_words > 150000;

OUTPUT



4. CALCULATING RESULTS WITH CASE

PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,

   type TEXT, minutes

   INTEGER, calories

   INTEGER,

   heart_rate INTEGER);


INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100,
110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30,
105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30,
70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25,
72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);


SELECT * FROM exercise_logs;


SELECT COUNT(*) FROM exercise_logs WHERE heart_rate > 220 - 30;

/* 50-90% of max*/


SELECT COUNT(*) FROM exercise_logs WHERE heart_rate
>= ROUND(0.50 * (220-30))
   AND heart_rate <= ROUND(0.90 * (220-30));
```

```
/* CASE */
SELECT type, heart_rate, CASE
        WHEN heart_rate > 220-30 THEN "above max"
        WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target"

        WHEN heart_rate > ROUND(0.50 * (220-30)) THEN "within target"

        ELSE "below target"

    END as "hr_zone" FROM
exercise_logs;



SELECT COUNT(*), CASE
        WHEN heart_rate > 220-30 THEN "above max"
        WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target"

        WHEN heart_rate > ROUND(0.50 * (220-30)) THEN "within target"

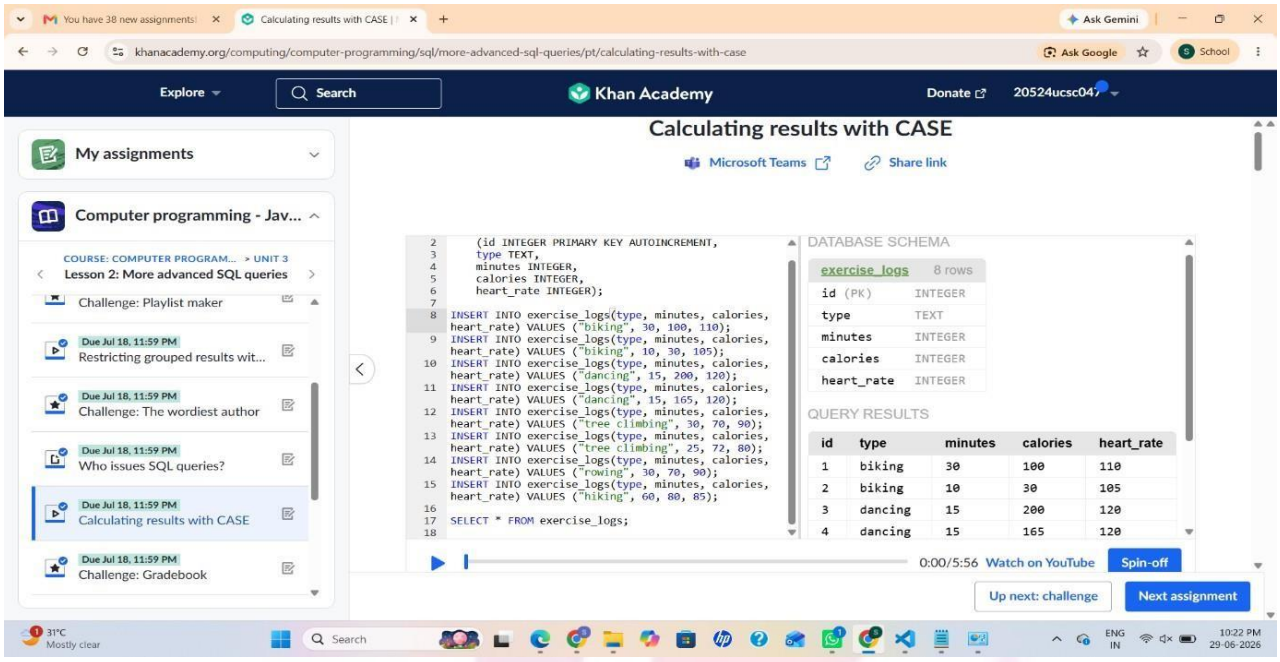
        ELSE "below target"

    END as "hr_zone" FROM

exercise_logs GROUP BY

hr_zone;
```

OUTPUT


Calculating results with CASE
Microsoft Teams
Share link
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 2: More advanced SQL queries
Challenge: Playlist maker
Restricting grouped results wit...
Challenge: The wordiest author
Who issues SQL queries?
Calculating results with CASE
Challenge: Gradebook
DATABASE SCHEMA
exercise_logs 8 rows
id (PK) INTEGER
type TEXT
minutes INTEGER
calories INTEGER
heart_rate INTEGER
QUERY RESULTS
id type minutes calories heart_rate
1 biking 30 100 110
2 biking 10 30 105
3 dancing 15 200 120
4 dancing 15 165 120
0:00/5:56 Watch on YouTube
Spin-off
Up next: challenge
Next assignment

5.GRADEBOOK

PROGRAM

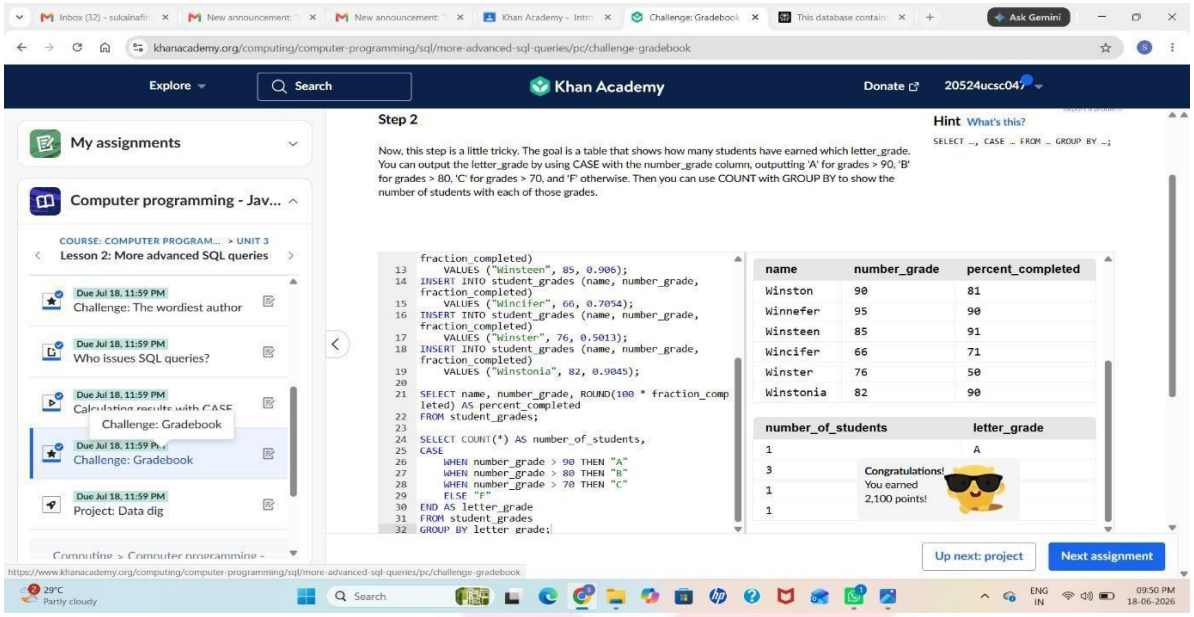
```
CREATE TABLE student_grades ( id INTEGER
   PRIMARY KEY AUTOINCREMENT, name
   TEXT, number_grade INTEGER,
   fraction_completed REAL);


INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winstonia", 82, 0.9045);


SELECT name, number_grade, ROUND(100 * fraction_completed) AS percent_completed
FROM student_grades;


SELECT COUNT(*) AS number_of_students, CASE
   WHEN number_grade > 90 THEN "A"
WHEN number_grade > 80 THEN "B" WHEN
number_grade > 70 THEN "C" ELSE "F" END
AS letter_grade FROM student_grades
GROUP BY letter_grade;
```

OUTPUT


Explore
Search
Khan Academy
Donate
My assignments
Computer programming - Jav...
Lesson 2: More advanced SQL queries
Challenge: The wordiest author
Who issues SQL queries?
Challenge: Gradebook
Project: Data dig
Step 2
Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.
Hint
What's this?
SELECT ..., CASE ... FROM ... GROUP BY ...;
name | number_grade | percent_completed
Winston | 90 | 81
Winnefer | 95 | 90
Winsteen | 85 | 91
Wincifer | 66 | 71
Winster | 76 | 50
Winstonia | 82 | 90
number_of_students | letter_grade
1 | A
3
1
1
Congratulations!
You earned
2,100 points!
Up next: project
Next assignment

6. PROJECT: DATA DIG

PROGRAM

```
CREATE TABLE astronauts ( Name
  TEXT PRIMARY KEY, Year
  INTEGER,
  GroupNum INTEGER, Status
  TEXT, Birth_Date TEXT,
  Birth_Place TEXT, Gender
  TEXT, Alma_Mater TEXT,
  Undergraduate_Major TEXT,
  Graduate_Major TEXT, Military_Rank
  TEXT, Military_Branch TEXT,
  Space_Flights INTEGER,
  Space_Flight_hr INTEGER,
  Space_Walks INTEGER,
  Space_Walks_hr REAL, Missions
  TEXT,
  Death_Date TEXT,
  Death_Mission TEXT
);



SELECT
  Gender,
  AVG(Space_Flights) AS avg_flights, MAX(Space_Flights)
  AS max_flights,

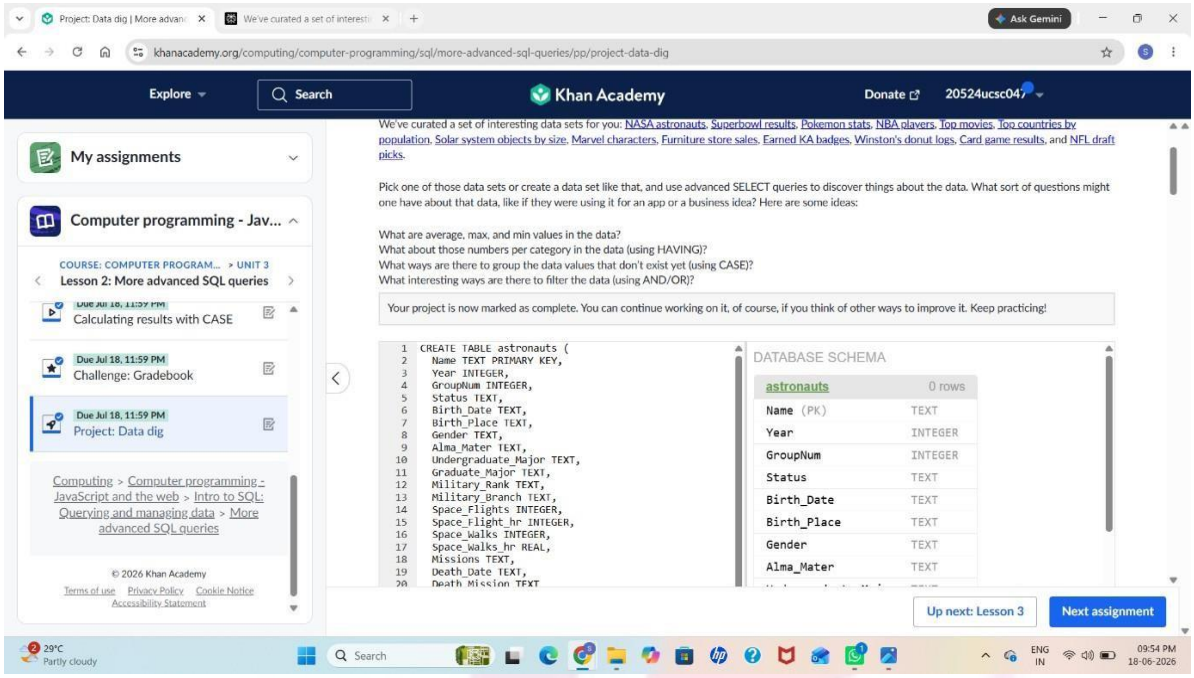



  MIN(Space_Flights) AS min_flights FROM

astronauts

GROUP BY Gender;
```

OUTPUT


Project: Data dig | More advanc
We've curated a set of interesti
Ask Gemini
khanacademy.org/computing/computer-programming/sql/more-advanced-sql-queries/pp/project-data-dig
Explore
Search
Khan Academy
Donate
20524ucsc047
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 2: More advanced SQL queries
Calculating results with CASE
Due Jul 18, 11:59 PM
Challenge: Gradebook
Due Jul 18, 11:59 PM
Project: Data dig
Computing > Computer programming - JavaScript and the web > Intro to SQL: Querying and managing data > More advanced SQL queries
© 2026 Khan Academy
Terms of use Privacy Policy Cookie Notice
Accessibility Statement
We've curated a set of interesting data sets for you: NASA astronauts, Superbowl results, Pokemon stats, NBA players, Top movies, Top countries by population, Solar system objects by size, Marvel characters, Furniture store sales, Earned KA badges, Winston's donut logs, Card game results, and NFL draft picks.
Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:
What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
CREATE TABLE astronauts (
Name TEXT PRIMARY KEY,
Year INTEGER,
GroupNum INTEGER,
Status TEXT,
Birth_Date TEXT,
Birth_Place TEXT,
Gender TEXT,
Alma_Mater TEXT,
Undergraduate_Major TEXT,
Graduate_Major TEXT,
Military_Rank TEXT,
Military_Branch TEXT,
Space_Flights INTEGER,
Space_Flight_hr INTEGER,
Space_Walks INTEGER,
Space_Walks_hr REAL,
Missions TEXT,
Death_Date TEXT,
Death_Mission TEXT
DATABASE SCHEMA
astronauts 0 rows
Name (PK) TEXT
Year INTEGER
GroupNum INTEGER
Status TEXT
Birth_Date TEXT
Birth_Place TEXT
Gender TEXT
Alma_Mater TEXT
Up next: Lesson 3
Next assignment
29°C
Partly cloudy
Search
ENG
IN
09:54 PM
18-06-2026

7. JOINING RELATED TABLES

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY

   KEY, first_name TEXT, last_name TEXT, email

   TEXT, phone TEXT, birthdate TEXT);


INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
   ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
   VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04");


CREATE TABLE student_grades (id INTEGER PRIMARY KEY,

   student_id INTEGER, test TEXT,

   grade INTEGER);


INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Chemistry", 85);


INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Chemistry", 95);
SELECT * FROM student_grades;


/* cross join */
SELECT * FROM student_grades, students;

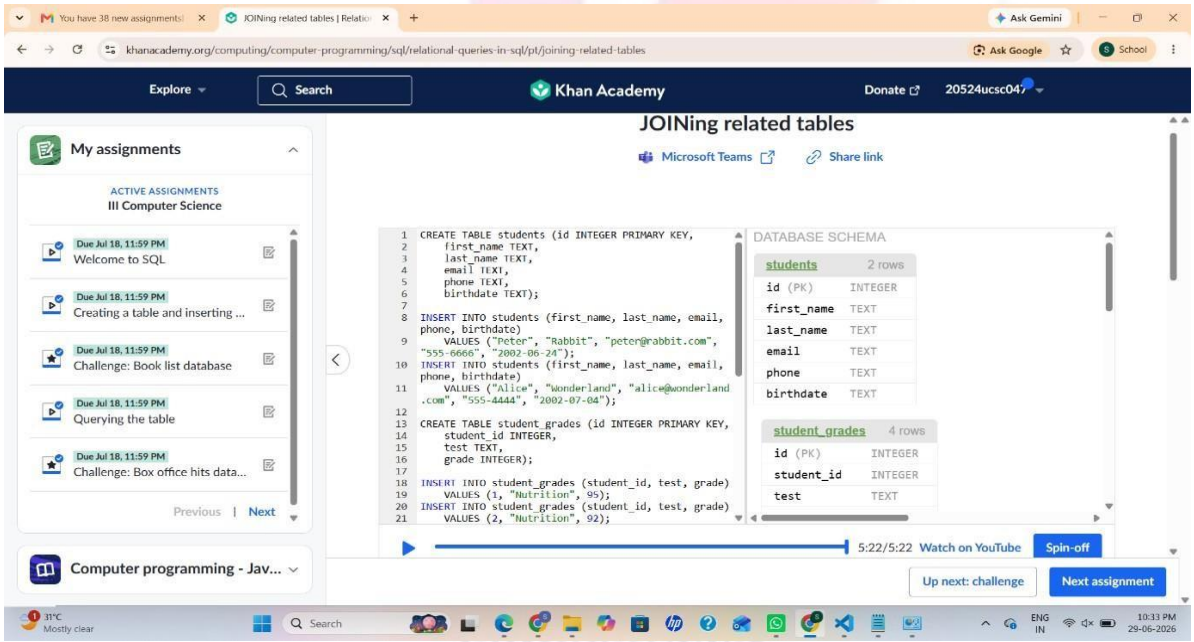

/* implicit inner join */
SELECT * FROM student_grades, students WHERE
```

```
    student_grades.student_id = students.id;


/* explicit inner join - JOIN */
SELECT students.first_name, students.last_name, students.email, student_grades.test,
student_grades.grade FROM students
    JOIN student_grades
    ON students.id = student_grades.student_id WHERE

    grade > 90;
```

OUTPUT



8.BOBBY'S HOBBIES

PROGRAM

```
CREATE TABLE persons ( id INTEGER
   PRIMARY KEY AUTOINCREMENT, name
   TEXT, age INTEGER
);


INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);


CREATE TABLE hobbies ( id INTEGER
   PRIMARY KEY AUTOINCREMENT, person_id
   INTEGER, name TEXT
);


INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");


 INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
 INSERT INTO hobbies (person_id, name) VALUES (4, "coding");

 INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");

 INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");  INSERT INTO persons
(name, age) VALUES ("Aisha Khan", 20);
INSERT INTO hobbies (person_id, name) VALUES (6, "coding");
```

SELECT persons.name, hobbies.name FROM

persons

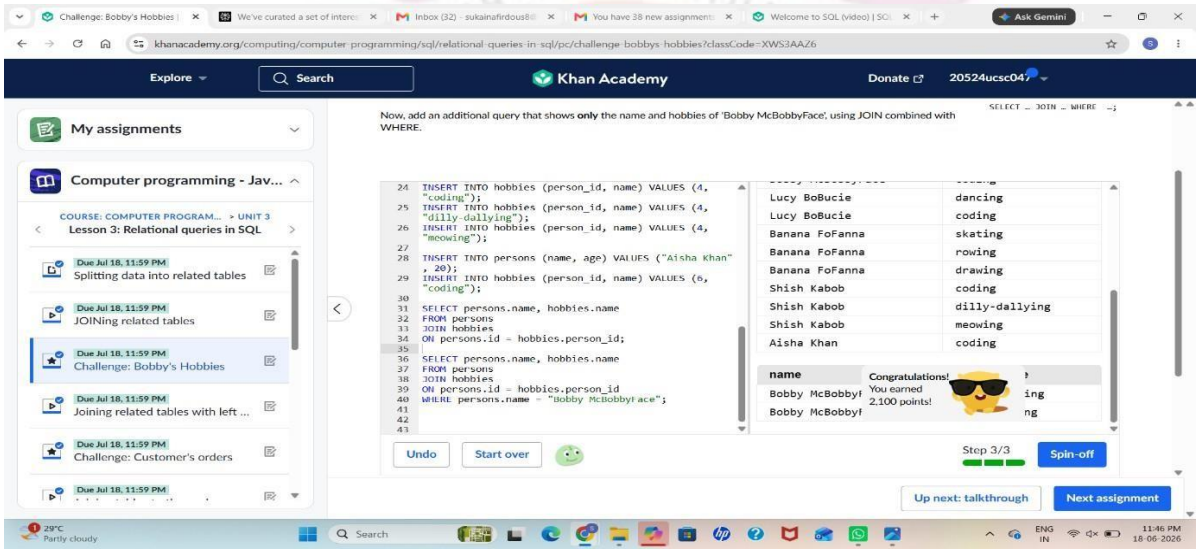
JOIN hobbies
ON persons.id = hobbies.person_id;

SELECT persons.name, hobbies.name FROM

persons

JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";

OUTPUT



9. JOINING RELATED TABLES WITH LEFT OUTER JOINS

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY

   KEY, first_name TEXT, last_name TEXT, email

   TEXT, phone TEXT, birthdate TEXT);


INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
   ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
   VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04");


CREATE TABLE student_grades (id INTEGER PRIMARY KEY,

   student_id INTEGER, test TEXT,

   grade INTEGER);


INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Chemistry", 85);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Chemistry", 95);


CREATE TABLE student_projects (     id INTEGER

PRIMARY KEY, student_id INTEGER, title TEXT);

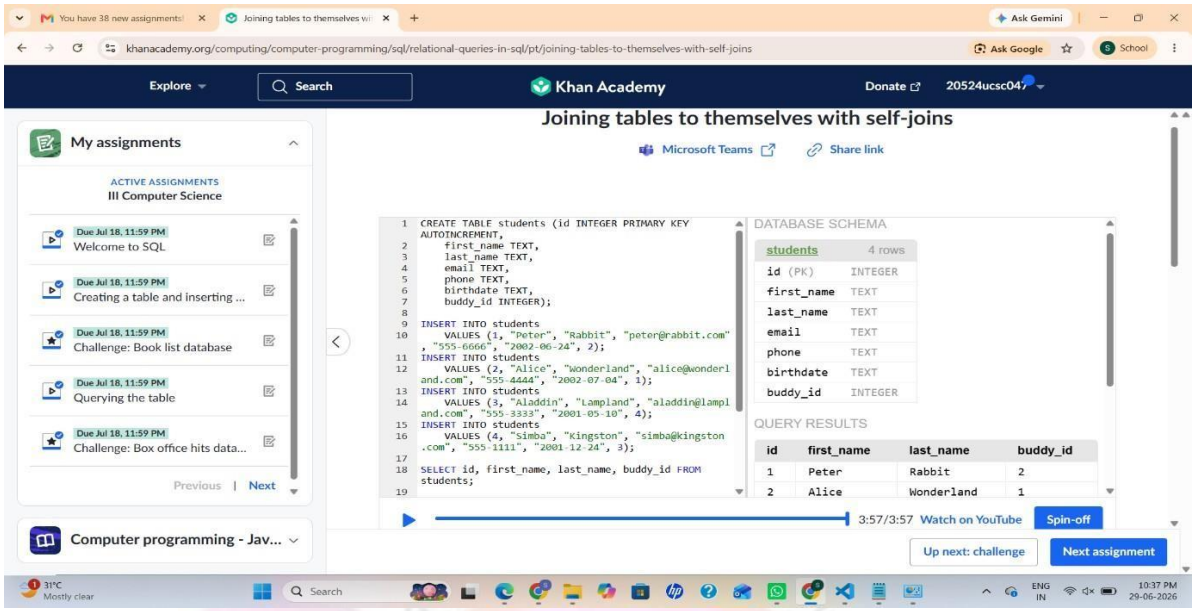

INSERT INTO student_projects (student_id, title) VALUES
   (1,"Carrotapault");


/* outer join */
```

```
SELECT students.first_name, students.last_name, student_projects.title

FROM students
    LEFT OUTER JOIN student_projects
    ON students.id = student_projects.student_id;
```

OUTPUT



10. CUSTOMER'S ORDERS

PROGRAM

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT, email

    TEXT);


INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");


CREATE TABLE orders ( id INTEGER PRIMARY

    KEY AUTOINCREMENT, customer_id

    INTEGER, item TEXT,

    price REAL);


      INSERT INTO orders (customer_id, item, price)
  VALUES (1, "Sonic Screwdriver", 1000.00); INSERT
           INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00); INSERT
INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);


SELECT customers.name, customers.email, orders.item, orders.price
 FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;


SELECT customers.name, customers.email, SUM(orders.price) AS total_spent FROM

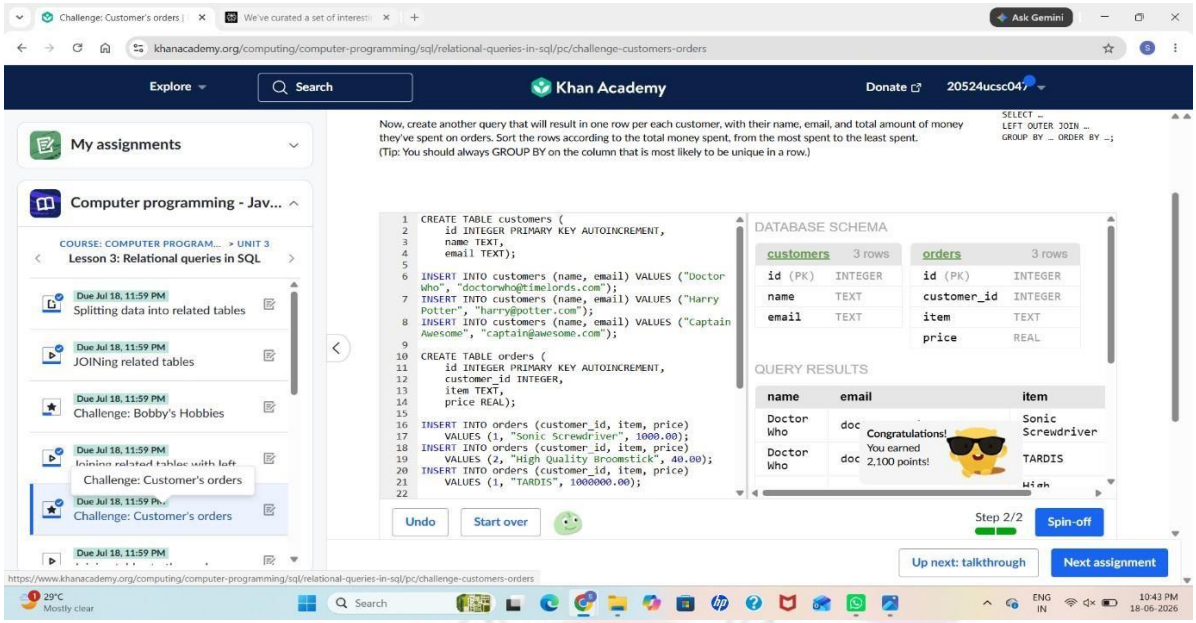
customers

LEFT OUTER JOIN orders
```

ON customers.id = orders.customer_id
GROUP BY customers.id, customers.name, customers.email ORDER

BY total_spent DESC;

OUTPUT

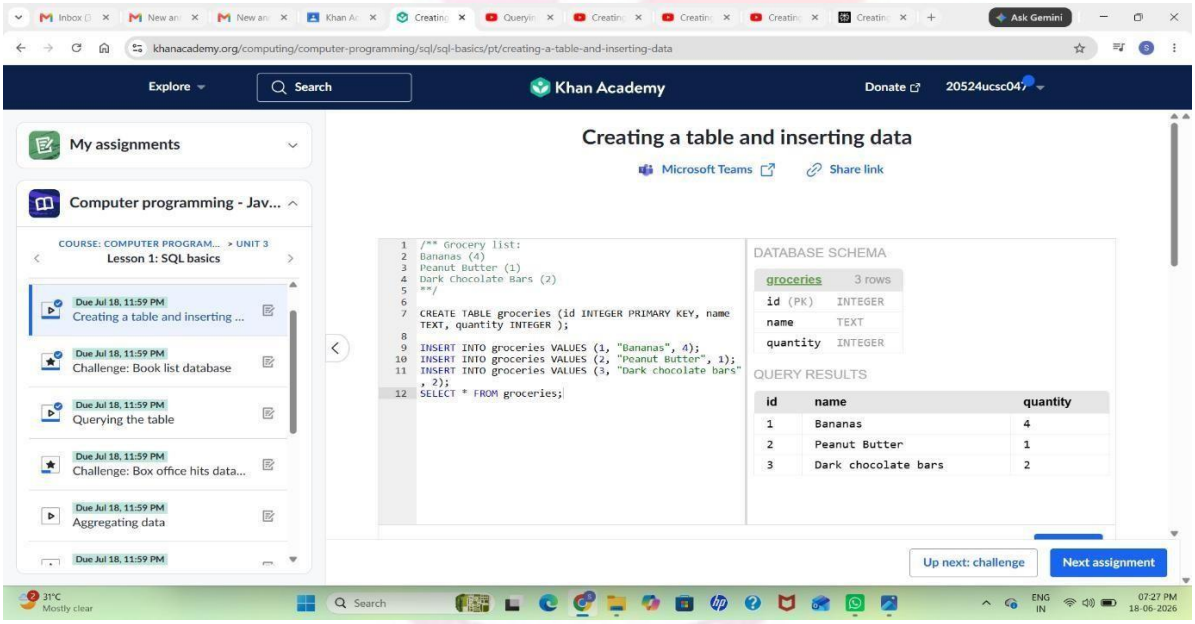


1. CREATING A TABLE AND INSERTING DATA

PROGRAM

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER );
INSERT INTO groceries VALUES (1, "Bananas", 4);
INSERT INTO groceries VALUES (2, "Peanut Butter", 1);
INSERT INTO groceries VALUES (3, "Dark chocolate
bars", 2);
SELECT * FROM groceries;
```

OUTPUT



12. BOOK LIST DATABASE

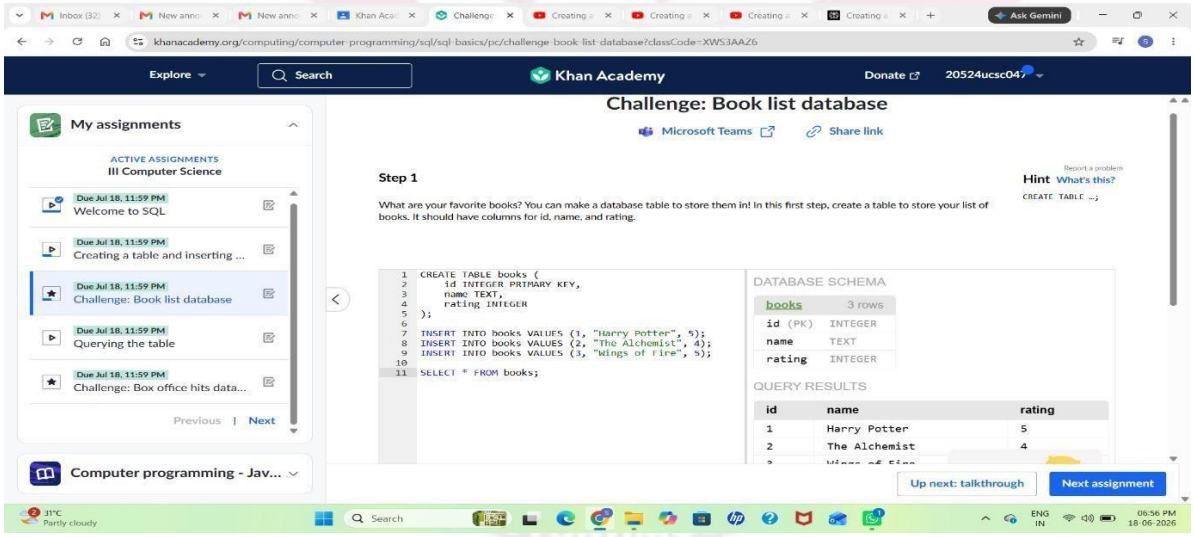
PROGRAM

```
CREATE TABLE books ( id
    INTEGER PRIMARY KEY,
    name TEXT, rating
    INTEGER
);

INSERT INTO books VALUES (1, "Harry Potter", 5);
INSERT INTO books VALUES (2, "The Alchemist", 4);
INSERT INTO books VALUES (3, "Wings of Fire", 5);

SELECT * FROM books;
```

OUTPUT



13. QUERYING THE TABLE

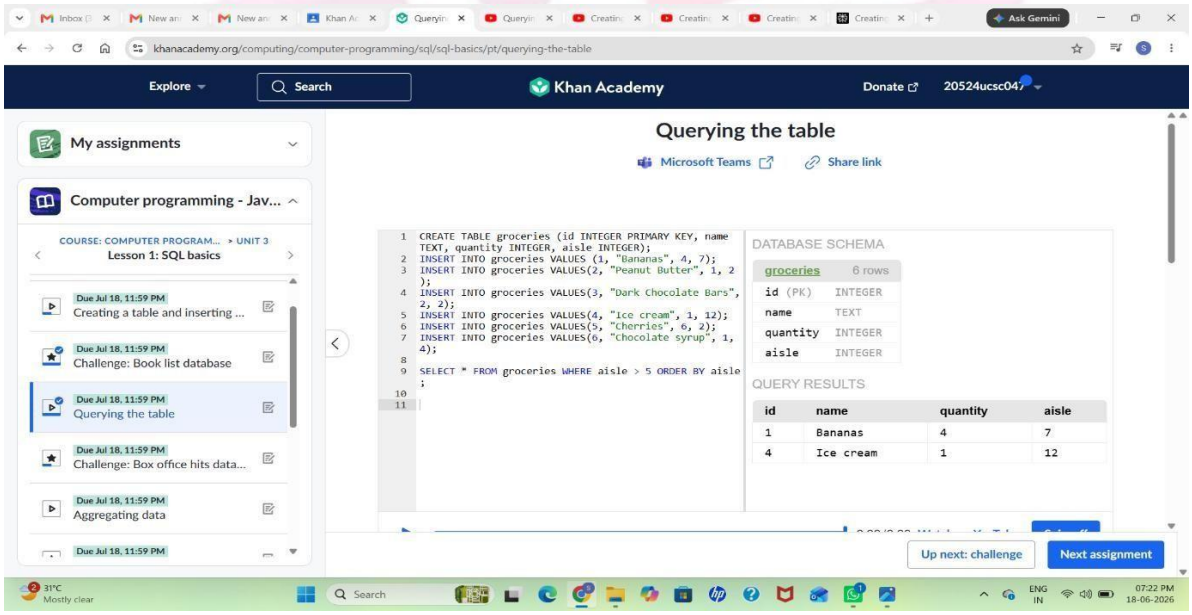
PROGRAM

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity
INTEGER, aisle INTEGER);
INSERT INTO groceries VALUES (1, "Bananas", 4, 7); INSERT
INTO groceries VALUES(2, "Peanut Butter", 1, 2);
INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2); INSERT
INTO groceries VALUES(4, "Ice cream", 1, 12);
INSERT INTO groceries VALUES(5, "Cherries", 6, 2); INSERT INTO

groceries VALUES(6, "Chocolate syrup", 1, 4);


SELECT * FROM groceries WHERE aisle > 5 ORDER BY aisle;
```

OUTPUT



14. BOX OFFICE HITS DATABASE

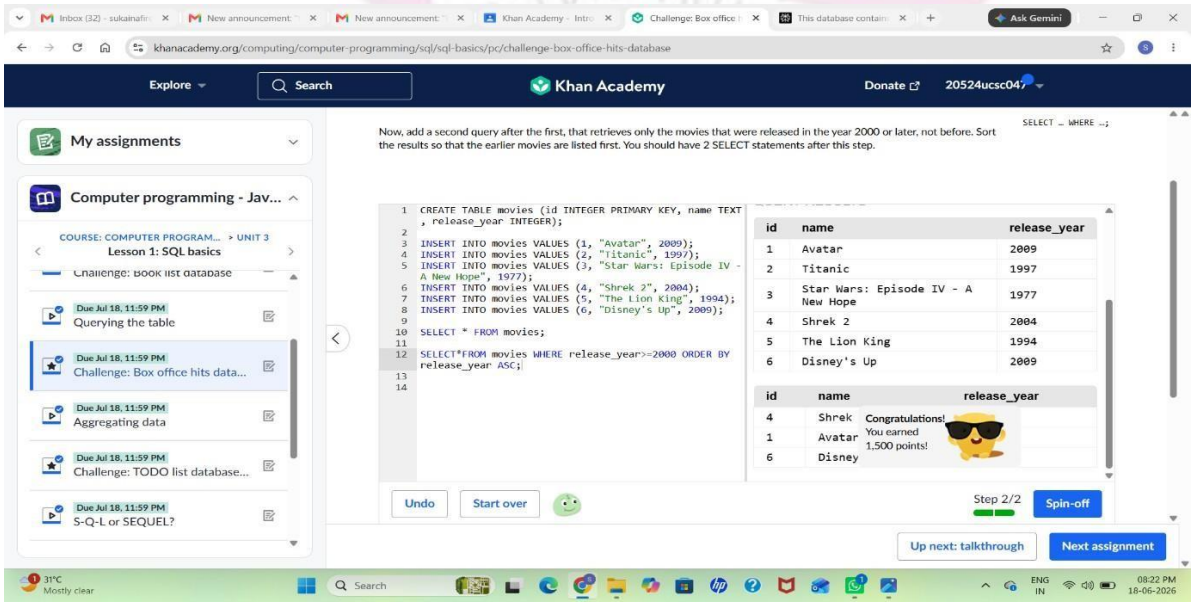
PROGRAM
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009); INSERT
INTO movies VALUES (2, "Titanic", 1997);
INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977); INSERT

INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994); INSERT
INTO movies VALUES (6, "Disney's Up", 2009);
SELECT * FROM movies;
SELECT*FROM movies WHERE release_year>=2000 ORDER BY release_year ASC;

OUTPUT



15. AGGREGATING DATA

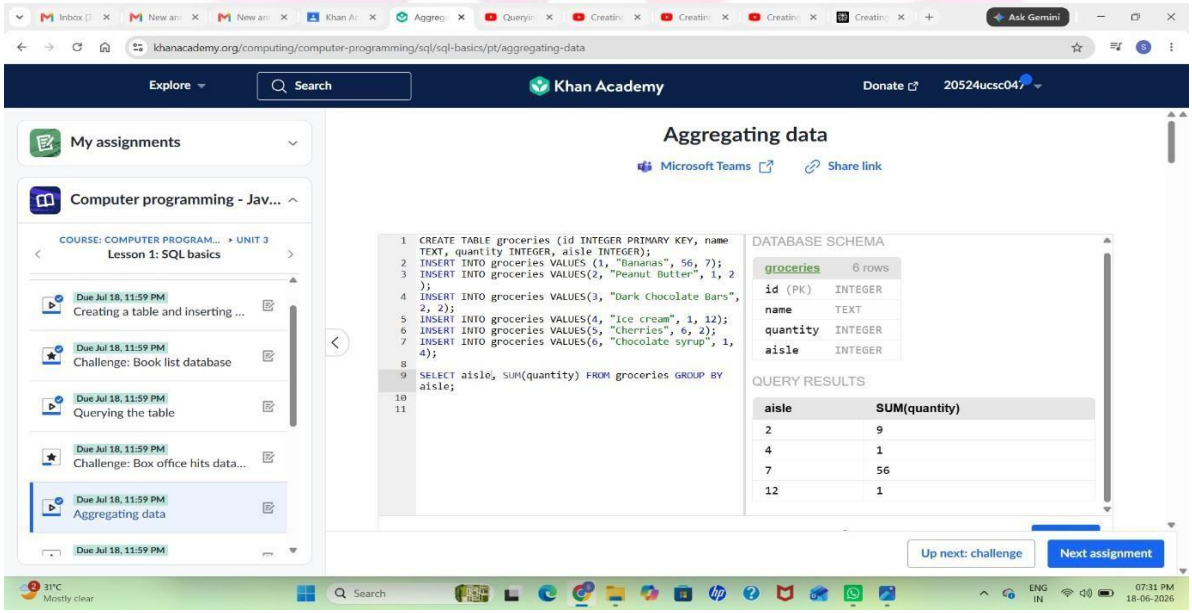
PROGRAM

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity
INTEGER, aisle INTEGER);
INSERT INTO groceries VALUES (1, "Bananas", 56, 7); INSERT
INTO groceries VALUES(2, "Peanut Butter", 1, 2);
INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2); INSERT
INTO groceries VALUES(4, "Ice cream", 1, 12);
INSERT INTO groceries VALUES(5, "Cherries", 6, 2); INSERT INTO

groceries VALUES(6, "Chocolate syrup", 1, 4);


SELECT aisle, SUM(quantity) FROM groceries GROUP BY aisle;
```

OUTPUT



16. TODO LIST DATABASE STATS

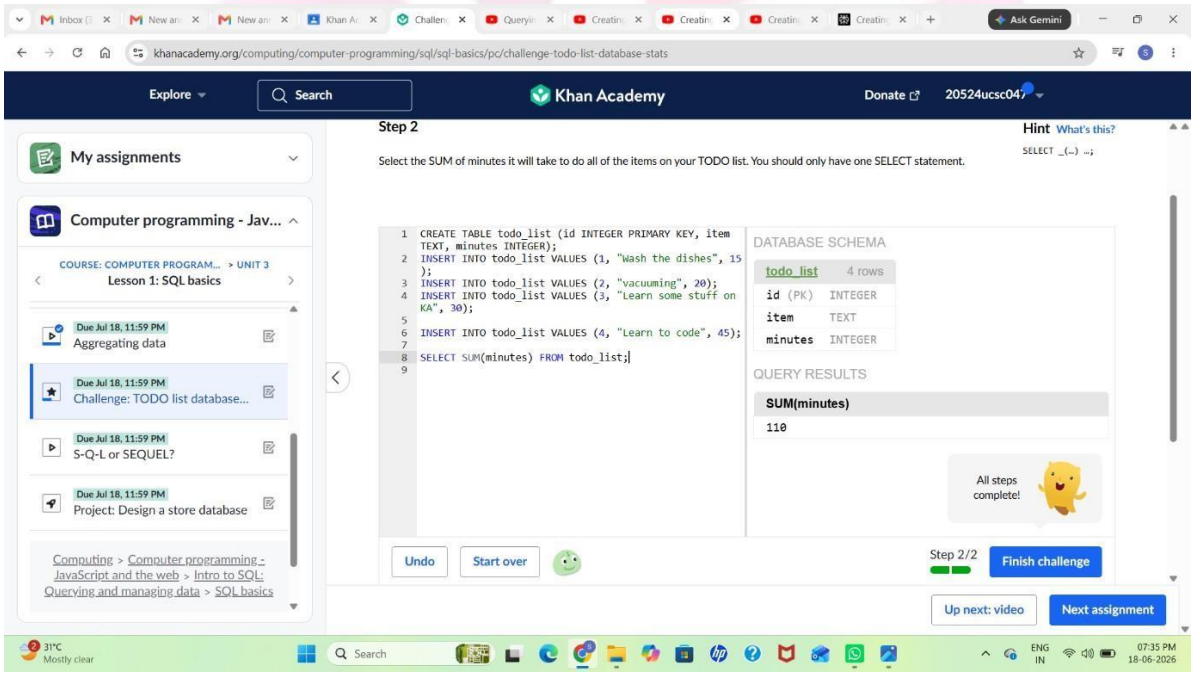
PROGRAM

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);
INSERT
INTO todo_list VALUES (1, "Wash the dishes", 15);
INSERT INTO todo_list VALUES (2, "vacuuming", 20);
INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);


INSERT INTO todo_list VALUES (4, "Learn to code", 45);


SELECT SUM(minutes) FROM todo_list;
```

OUTPUT



17. PROJECT: DESIGN A STORE DATABASE

PROGRAM

```
CREATE TABLE books ( id
   INTEGER PRIMARY KEY,
   name TEXT, author TEXT,
   genre TEXT, price REAL,
   stock
   INTEGER
);


INSERT INTO books VALUES (1, "Harry Potter 1", "J.K. Rowling", "Fantasy", 12.99, 50);

INSERT INTO books VALUES (2, "Harry Potter 2", "J.K. Rowling", "Fantasy", 13.99, 45);

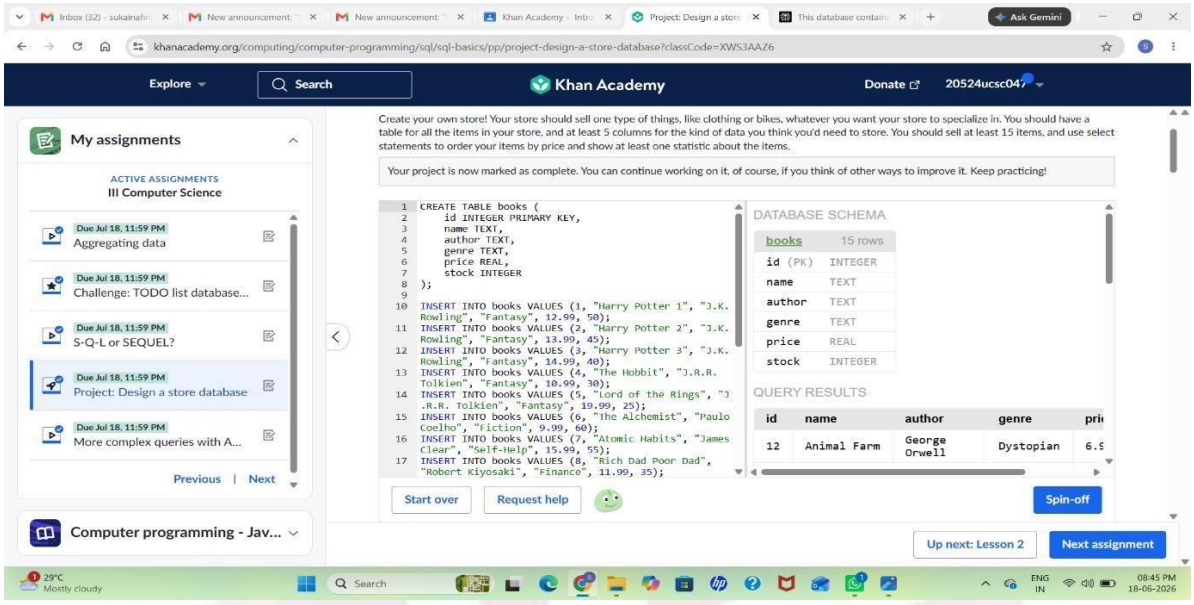
INSERT INTO books VALUES (3, "Harry Potter 3", "J.K. Rowling", "Fantasy", 14.99, 40);
INSERT INTO books VALUES (4, "The Hobbit", "J.R.R. Tolkien", "Fantasy", 10.99, 30);
INSERT INTO books VALUES (5, "Lord of the Rings", "J.R.R. Tolkien", "Fantasy", 19.99, 25);
INSERT INTO books VALUES (6, "The Alchemist", "Paulo Coelho", "Fiction", 9.99, 60);
INSERT
INTO books VALUES (7, "Atomic Habits", "James Clear", "Self-Help", 15.99, 55);
INSERT INTO books VALUES (8, "Rich Dad Poor Dad", "Robert Kiyosaki", "Finance", 11.99,
35);
INSERT INTO books VALUES (9, "Think and Grow Rich", "Napoleon Hill", "Finance", 8.99,
40);
INSERT INTO books VALUES (10, "Deep Work", "Cal Newport", "Self-Help", 13.99, 20);
INSERT INTO books VALUES (11, "1984", "George Orwell", "Dystopian", 7.99, 70);
INSERT INTO books VALUES (12, "Animal Farm", "George Orwell", "Dystopian", 6.99, 65);
INSERT INTO books VALUES (13, "The Great Gatsby", "F. Scott Fitzgerald", "Classic", 8.99,
30);
INSERT INTO books VALUES (14, "To Kill a Mockingbird", "Harper Lee", "Classic", 9.99,
25);
INSERT INTO books VALUES (15, "The Diary of a Young Girl", "Anne Frank", "Biography",
10.99, 50);


SELECT * FROM books ORDER BY price ASC;


SELECT MAX(price) FROM books; SELECT
MIN(price) FROM books; SELECT AVG(price)
```

FROM books;

OUTPUT



18. MORE COMPLEX QUERIES WITH AND/OR

PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,

   type TEXT, minutes

   INTEGER, calories

   INTEGER,

   heart_rate INTEGER);
```

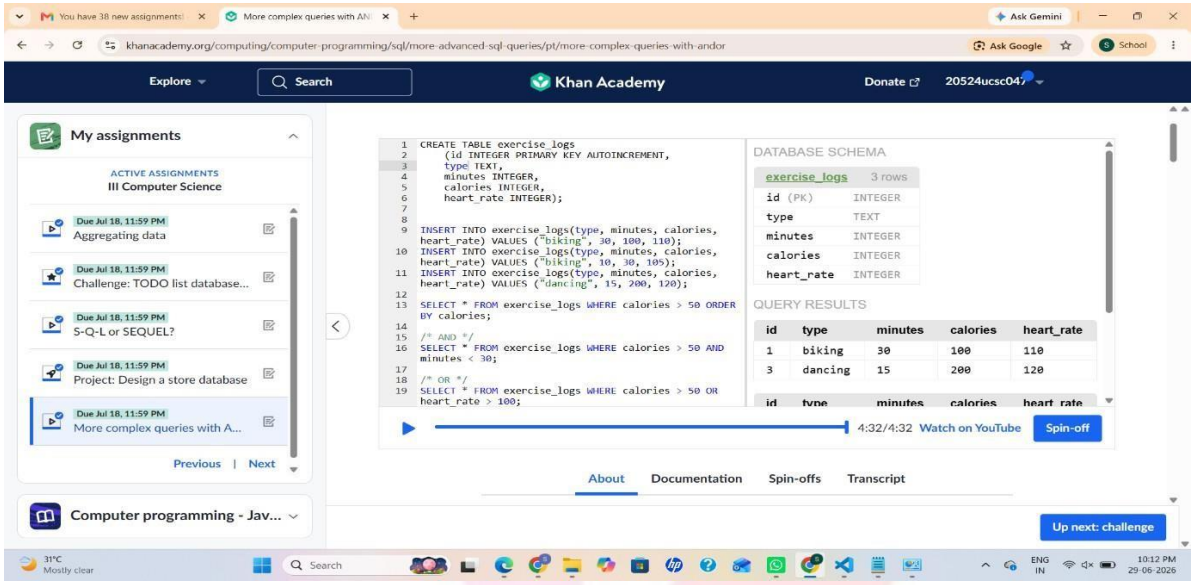
```
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100,
110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30,
105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);
```

```
SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;

/* AND */
SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;

/* OR */
SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
```

OUTPUT


Explore
Search
Khan Academy
Donate
20524ucsc047
My assignments
ACTIVE ASSIGNMENTS
III Computer Science
Due Jul 18, 11:59 PM
Aggregating data
Due Jul 18, 11:59 PM
Challenge: TODO list database...
Due Jul 18, 11:59 PM
S-Q-L or SEQUEL?
Due Jul 18, 11:59 PM
Project: Design a store database
Due Jul 18, 11:59 PM
More complex queries with A...
Previous | Next
Computer programming - Jav...
CREATE TABLE exercise_logs
(id INTEGER PRIMARY KEY AUTOINCREMENT,
type TEXT,
minutes INTEGER,
calories INTEGER,
heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;
/* AND */
SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;
/* OR */
SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
DATABASE SCHEMA
exercise_logs 3 rows
id (PK) INTEGER
type TEXT
minutes INTEGER
calories INTEGER
heart_rate INTEGER
QUERY RESULTS
id type minutes calories heart_rate
1 biking 30 100 110
3 dancing 15 200 120
4:32/4:32 Watch on YouTube
Spin-off
About
Documentation
Spin-offs
Transcript
Up next: challenge

19. KARAOKE SONG SELECTOR

PROGRAM

```
CREATE TABLE songs ( id
   INTEGER PRIMARY KEY,
   title TEXT, artist
   TEXT, mood TEXT,
   duration INTEGER,
   released
   INTEGER);

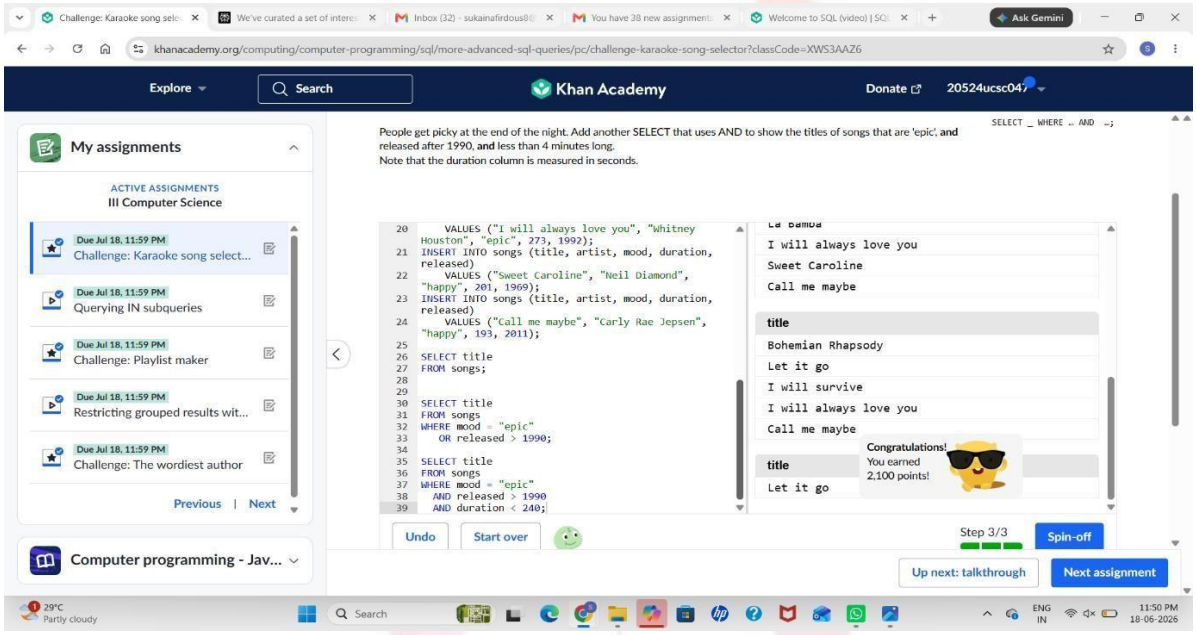

INSERT INTO songs (title, artist, mood, duration, released) VALUES
   ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released) VALUES
   ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released) VALUES ("I
   will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released) VALUES
   ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released) VALUES ("La
   Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992); INSERT
INTO songs (title, artist, mood, duration, released)
   VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969); INSERT
INTO songs (title, artist, mood, duration, released)
   VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);


SELECT title
FROM songs;


SELECT title FROM
songs
WHERE mood = "epic" OR
  released > 1990;
```

```
SELECT title FROM
songs
WHERE mood = "epic" AND
  released > 1990 AND duration <
  240;
```

OUTPUT



20. QUERYING IN SUBQUERIES

PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,

   type TEXT, minutes

   INTEGER, calories

   INTEGER,

   heart_rate INTEGER);



INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100,
110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30,
105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30,
70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25,
72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);



SELECT * FROM exercise_logs WHERE type = "biking" OR type = "hiking" OR type = "tree
climbing" OR type = "rowing";



/* IN */
SELECT * FROM exercise_logs WHERE type IN ("biking", "hiking", "tree climbing",
"rowing");




CREATE TABLE drs_favorites (id
INTEGER PRIMARY KEY,

   type TEXT, reason

   TEXT);
```

```
INSERT INTO drs_favorites(type, reason) VALUES ("biking", "Improves endurance and
flexibility.");
INSERT INTO drs_favorites(type, reason) VALUES ("hiking", "Increases cardiovascular
health.");


SELECT type FROM drs_favorites;


SELECT * FROM exercise_logs WHERE type IN ( SELECT

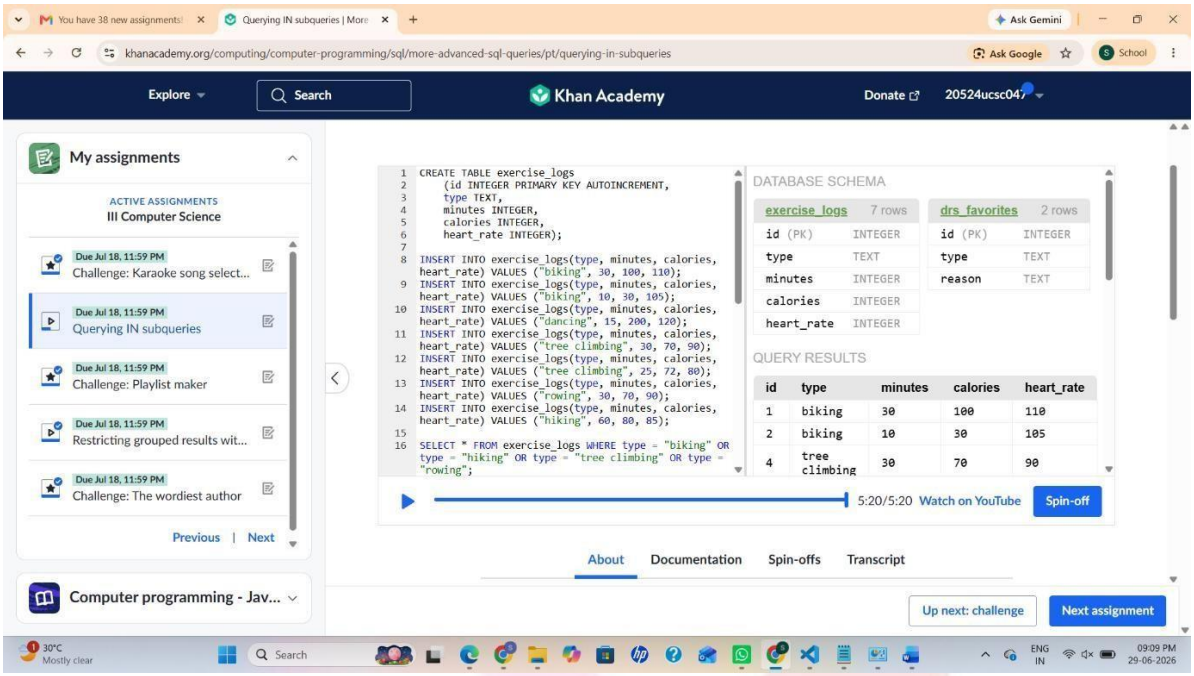
   type FROM drs_favorites);


SELECT * FROM exercise_logs WHERE type IN (
   SELECT type FROM drs_favorites WHERE reason = "Increases cardiovascular health");


/* LIKE */


SELECT * FROM exercise_logs WHERE type IN (
 SELECT type FROM drs_favorites WHERE reason LIKE "%cardiovascular%");
```

OUTPUT


You have 38 new assignments!
Querying IN subqueries | More
Ask Gemini
khanacademy.org/computing/computer-programming/sql/more-advanced-sql-queries/pt/querying-in-subqueries
Ask Google
School
Explore
Search
Khan Academy
Donate
20524ucsc047
My assignments
ACTIVE ASSIGNMENTS
III Computer Science
Due Jul 18, 11:59 PM
Challenge: Karaoke song select...
Due Jul 18, 11:59 PM
Querying IN subqueries
Due Jul 18, 11:59 PM
Challenge: Playlist maker
Due Jul 18, 11:59 PM
Restricting grouped results wit...
Due Jul 18, 11:59 PM
Challenge: The wordiest author
Previous | Next
Computer programming - Jav...
CREATE TABLE exercise_logs
(id INTEGER PRIMARY KEY AUTOINCREMENT,
type TEXT,
minutes INTEGER,
calories INTEGER,
heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);
SELECT * FROM exercise_logs WHERE type = "biking" OR type = "hiking" OR type = "tree climbing" OR type = "rowing";
DATABASE SCHEMA
exercise_logs 7 rows
id (PK) INTEGER
type TEXT
minutes INTEGER
calories INTEGER
heart_rate INTEGER
drs_favorites 2 rows
id (PK) INTEGER
type TEXT
reason TEXT
QUERY RESULTS
id type minutes calories heart_rate
1 biking 30 100 110
2 biking 10 30 105
4 tree climbing 30 70 90
5:20/5:20 Watch on YouTube
Spin-off
About
Documentation
Spin-offs
Transcript
Up next: challenge
Next assignment
30°C
Mostly clear
Search
ENG
IN
09:09 PM
29-06-2026

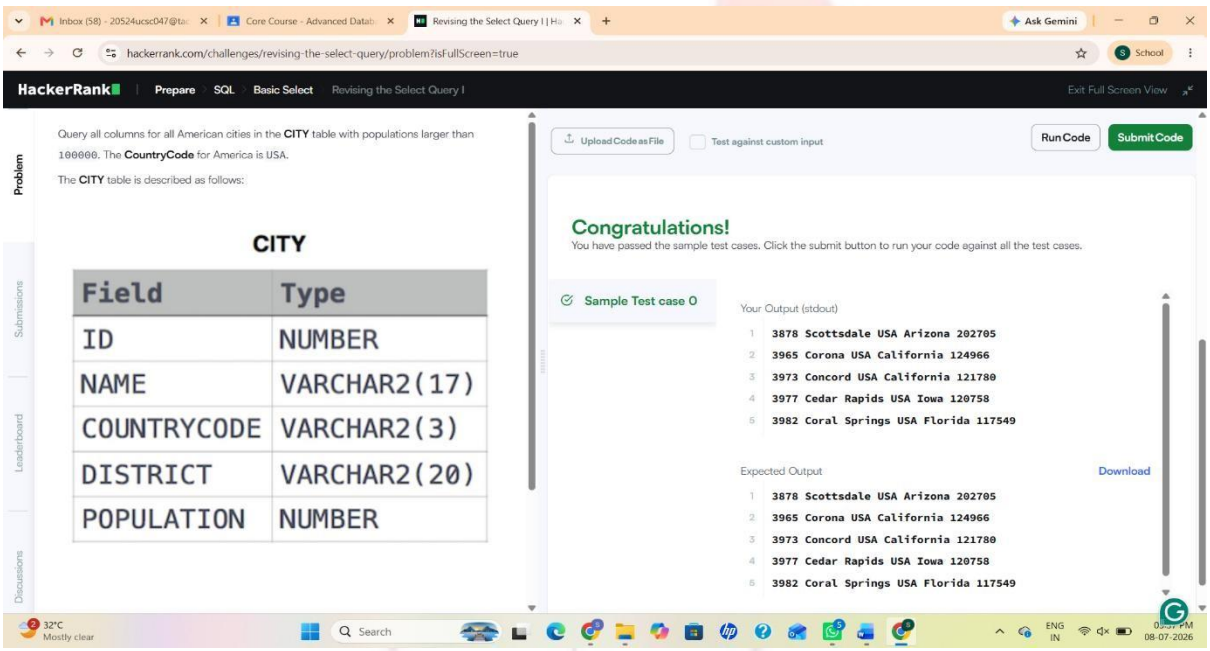
HACKER RANK

1.REVISING THE SELECT QUERY I

PROGRAM

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION >
100000;
```

OUTPUT

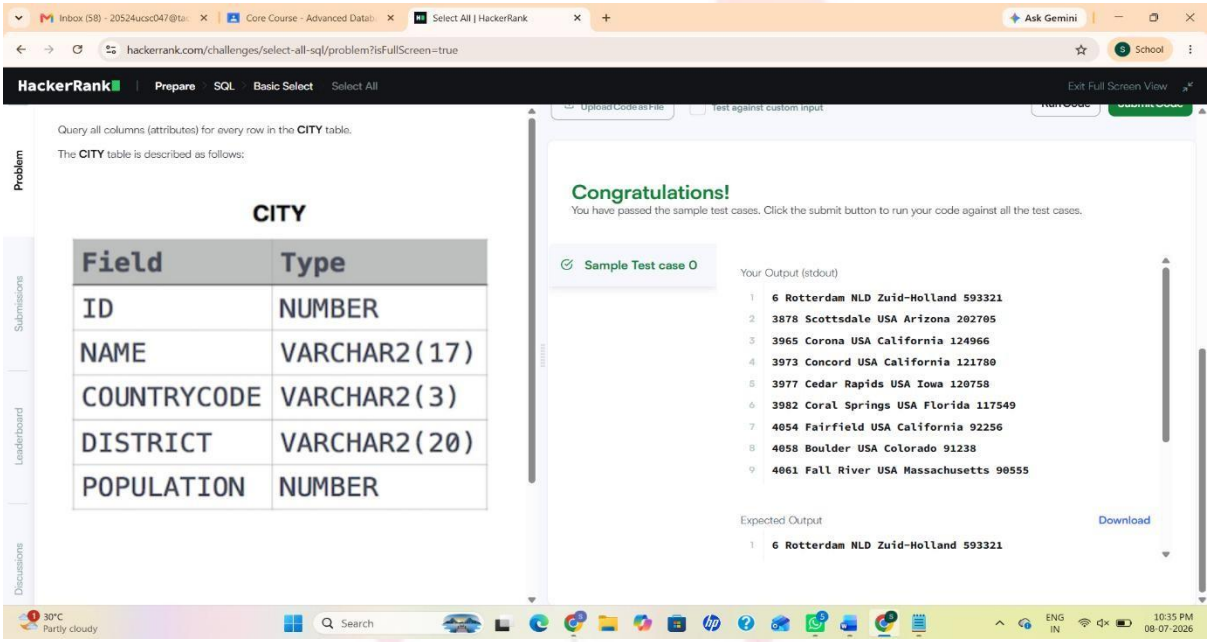


2.SELECT ALL

PROGRAM

```
SELECT t.*
FROM CITY t;
```

OUTPUT

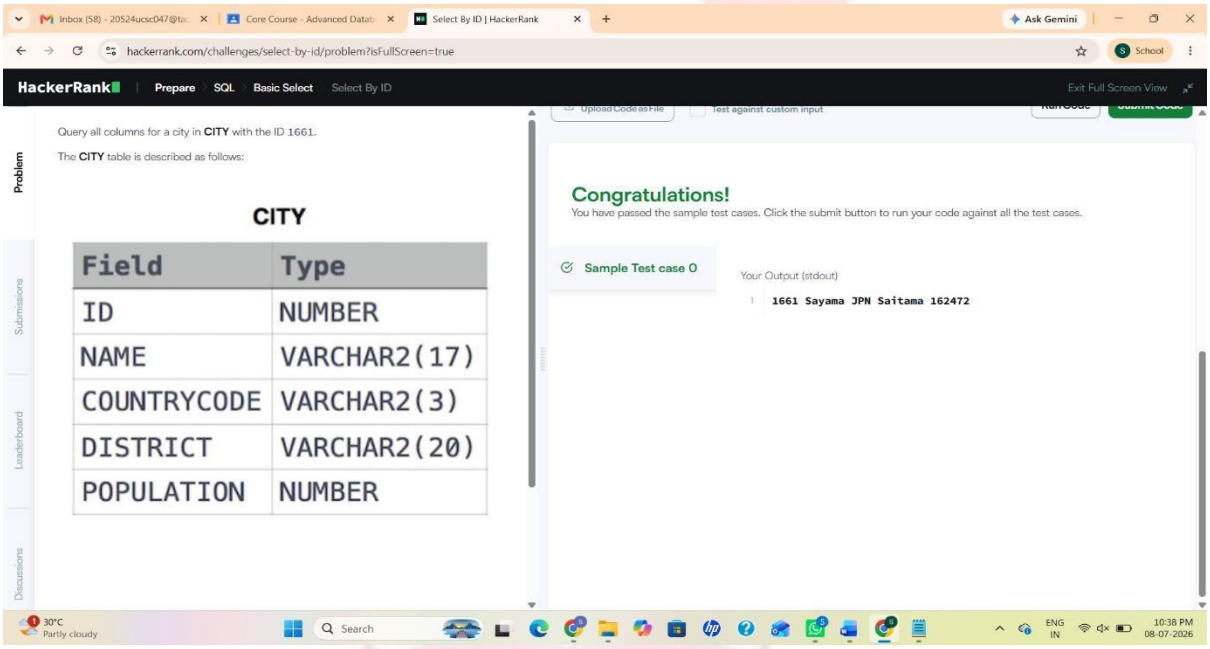


3.SELECT BY ID

PROGRAM

```
SELECT t.*
FROM CITY t
WHERE t.ID=1661;
```

OUTPUT

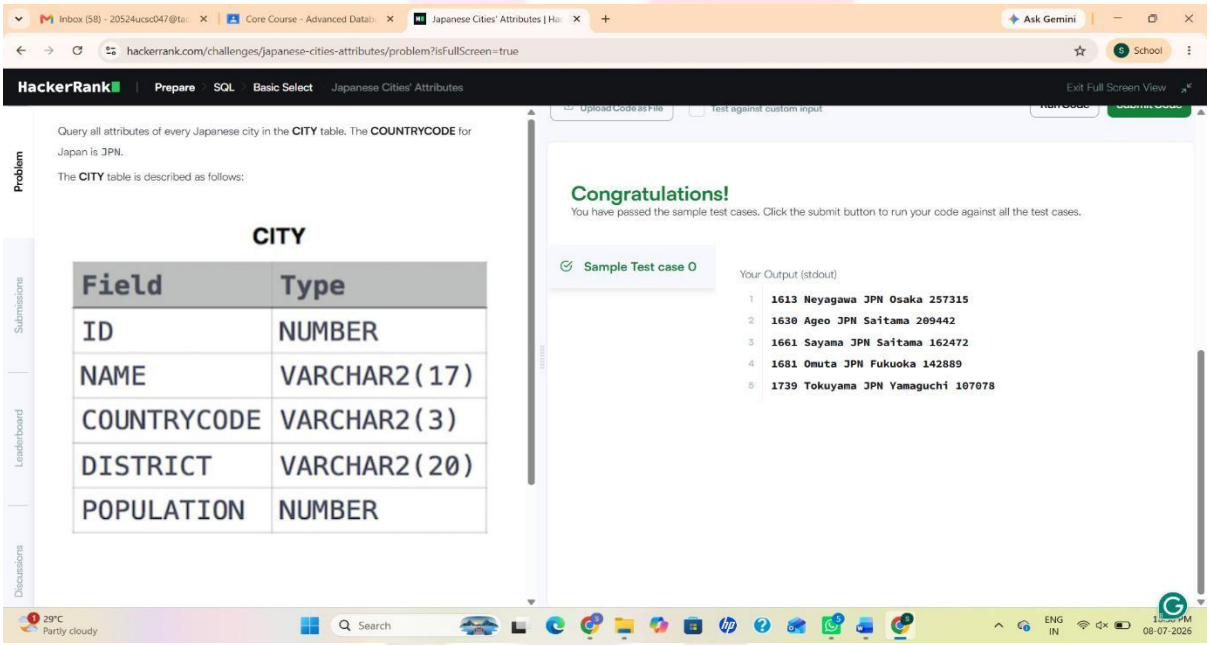


4. JAPANESE CITIES' ATTRIBUTES

program

```
SELECT t.*
FROM CITY t
WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT

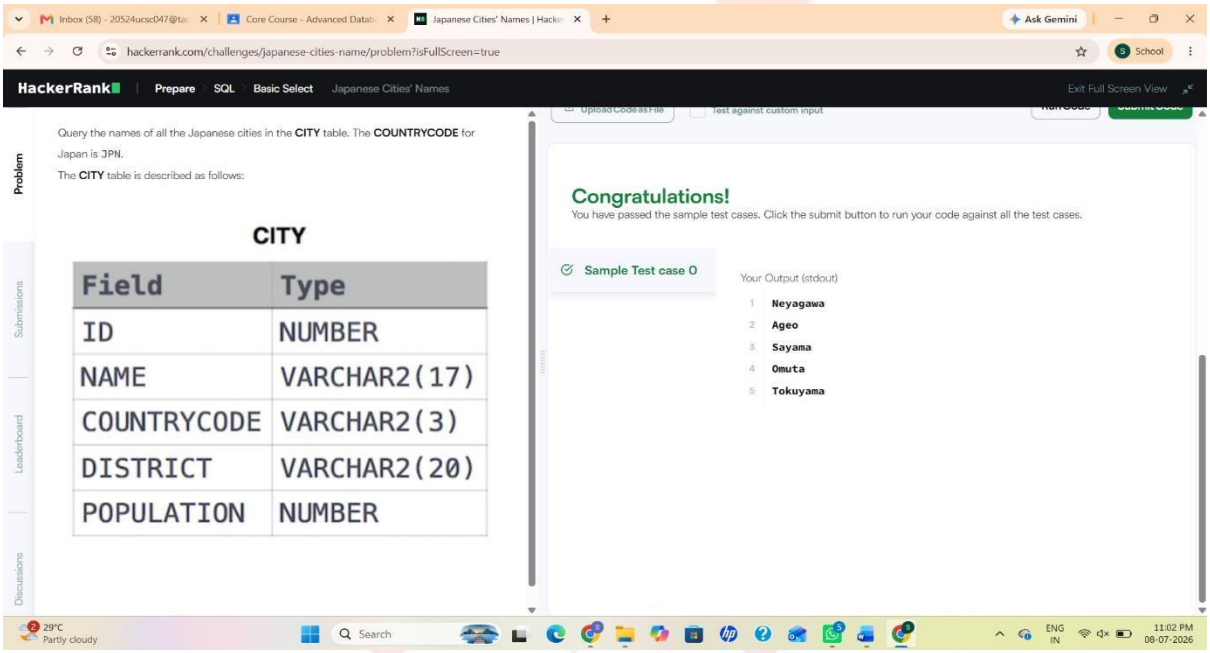


5.JAPANESE CITIES' NAMES

PROGRAM

```
SELECT t.NAME
FROM CITY t
WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT

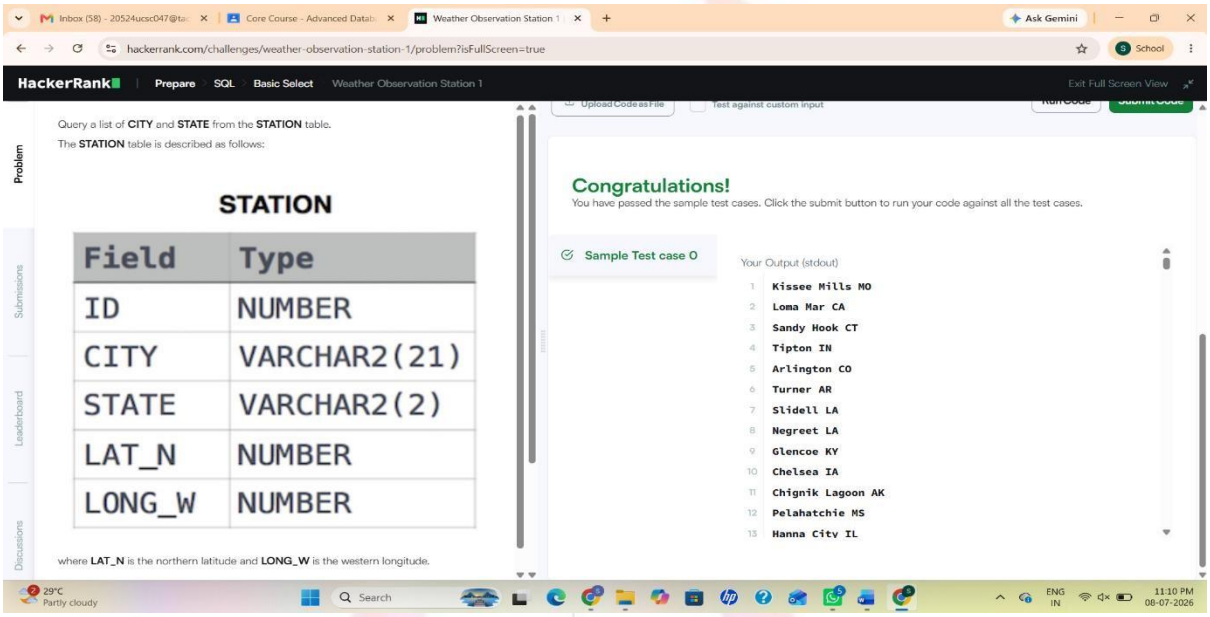


6. WEATHER OBSERVATION STATION 1

Prgram

```
SELECT t.CITY,t.STATE
FROM STATION t;
```

OUTPUT

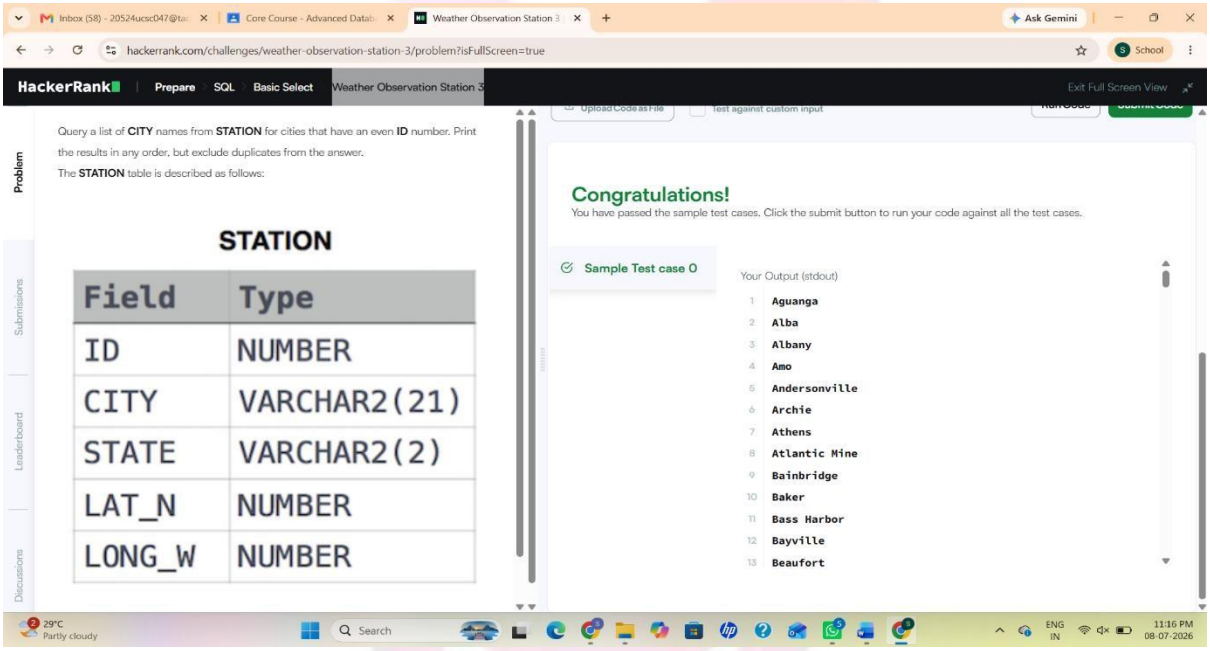


7. WEATHER OBSERVATION STATION 3

PROGRAM

```
SELECT DISTINCT t.CITY
FROM STATION t
WHERE MOD(t.ID,2) = 0;
```

OUTPUT



8 .WEATHER OBSERVATION STATION 4

PROGRAM

```
SELECT COUNT(t.CITY) - COUNT(DISTINCT t.CITY)
FROM STATION t;
```

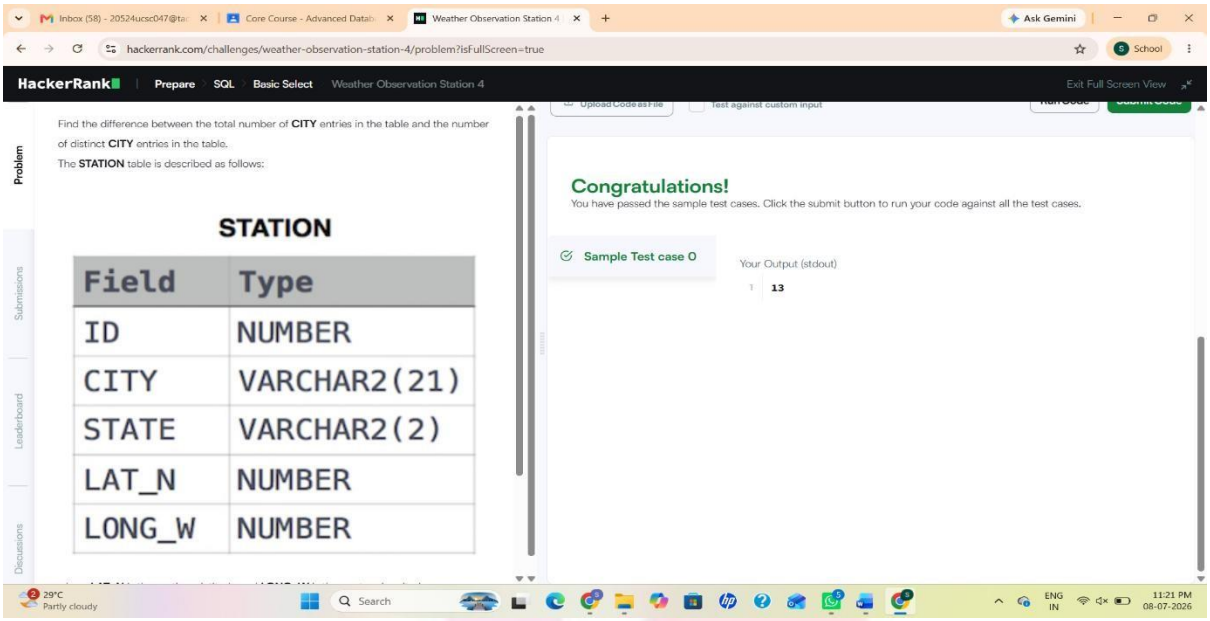
PROGRAM

SELECT COUNT(t.CITY) –

COUNT(DISTINCT t.CITY)

FROM STATION t;

OUTPUT


HackerRank
Prepare
SQL
Basic Select
Weather Observation Station 4
Exit Full Screen View
Problem
Submissions
Leaderboard
Discussions
Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table.
The STATION table is described as follows:
STATION
Field
Type
ID
NUMBER
CITY
VARCHAR2(21)
STATE
VARCHAR2(2)
LAT_N
NUMBER
LONG_W
NUMBER
Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.
Sample Test case 0
Your Output (stdout)
13

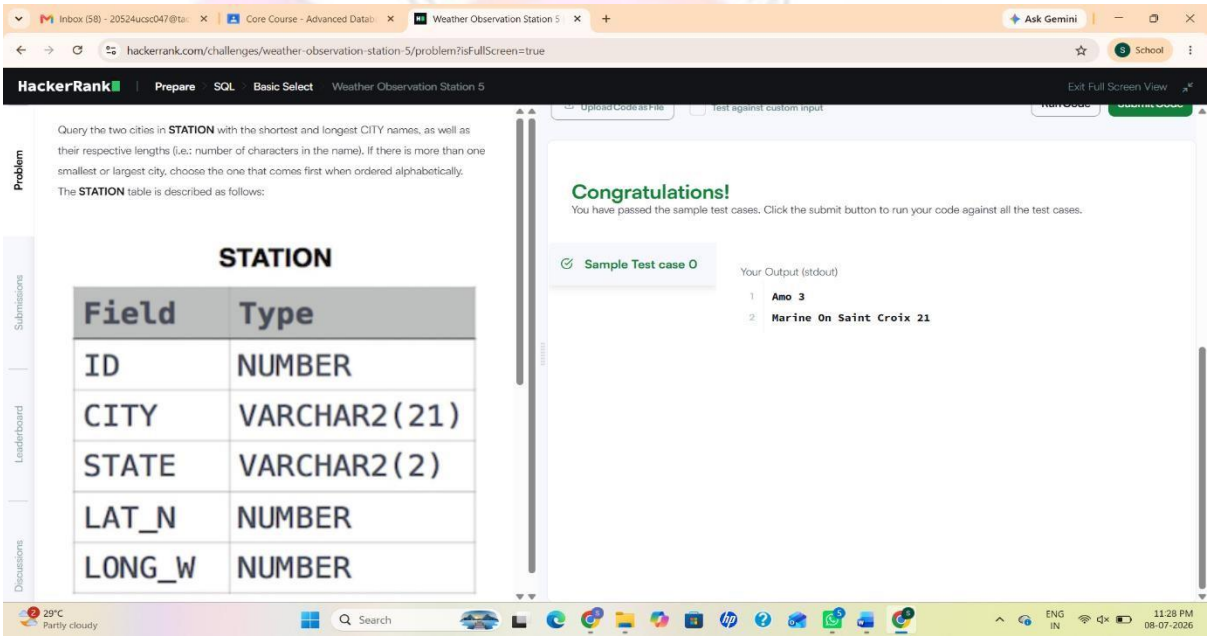
9. WEATHER OBSERVATION STATION 5

PROGRAM

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY), CITY
FETCH FIRST 1 ROW ONLY;
```

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY
FETCH FIRST 1 ROW ONLY;
```

OUTPUT

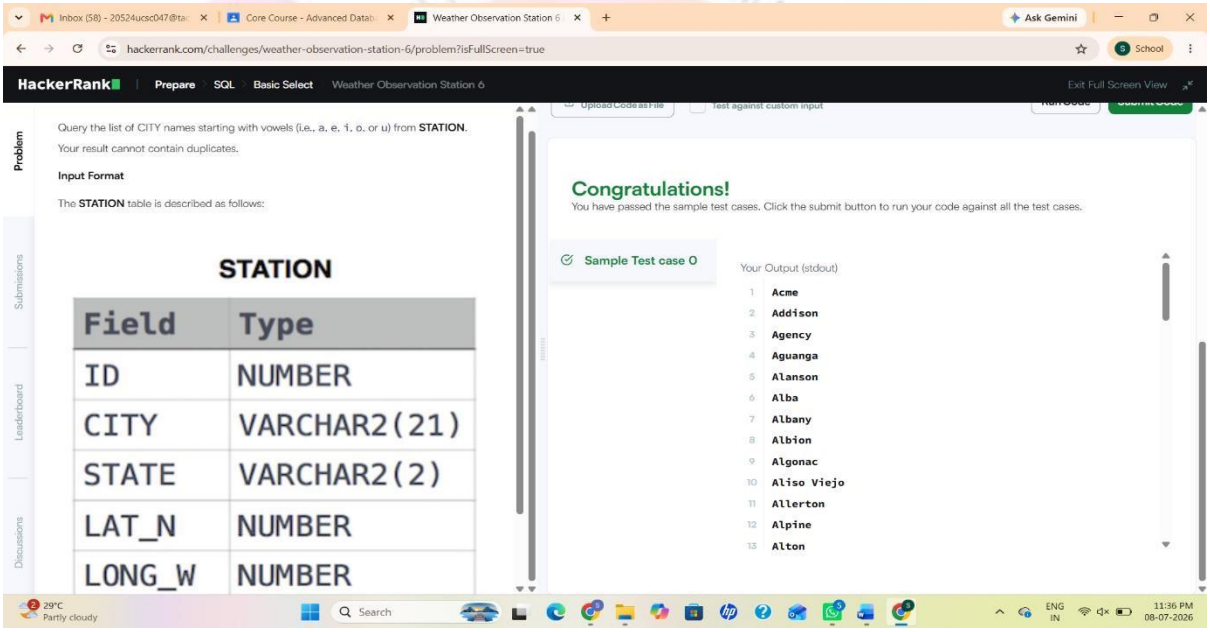


10. WEATHER OBSERVATION STATION 6

Program

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,
1,1)) IN
('A','E','I','O','U');
```

OUTPUT

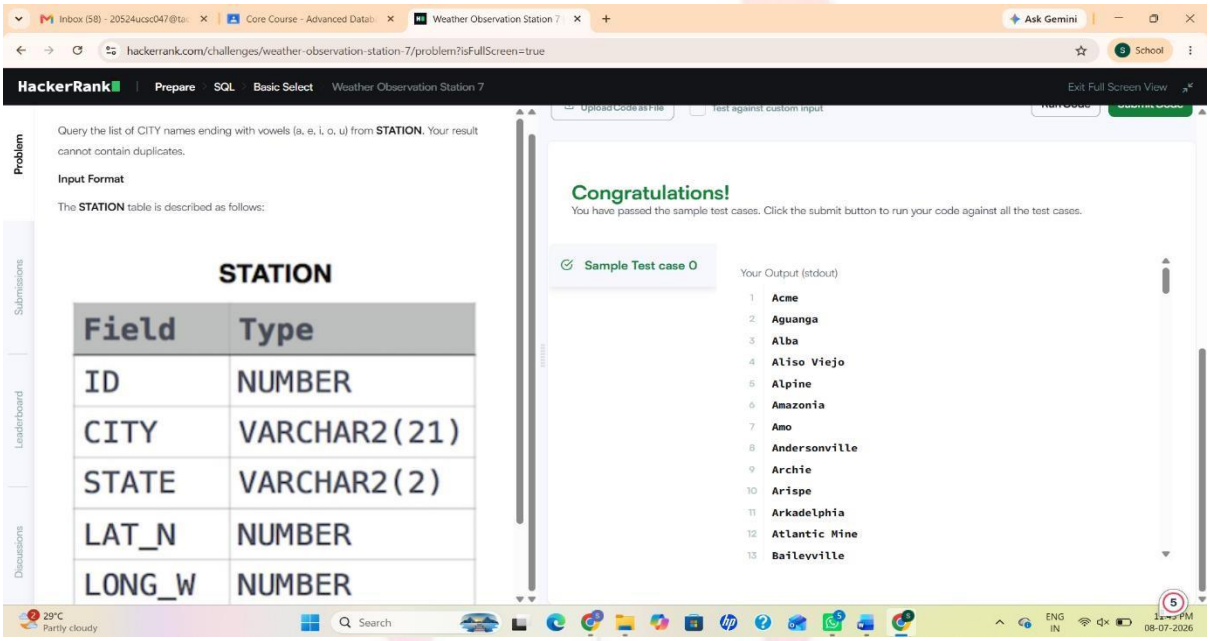


11. WEATHER OBSERVATION STATION 7

Program

```
SELECT DISTINCT CITY FROM STATION WHERE
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
```

OUTPUT

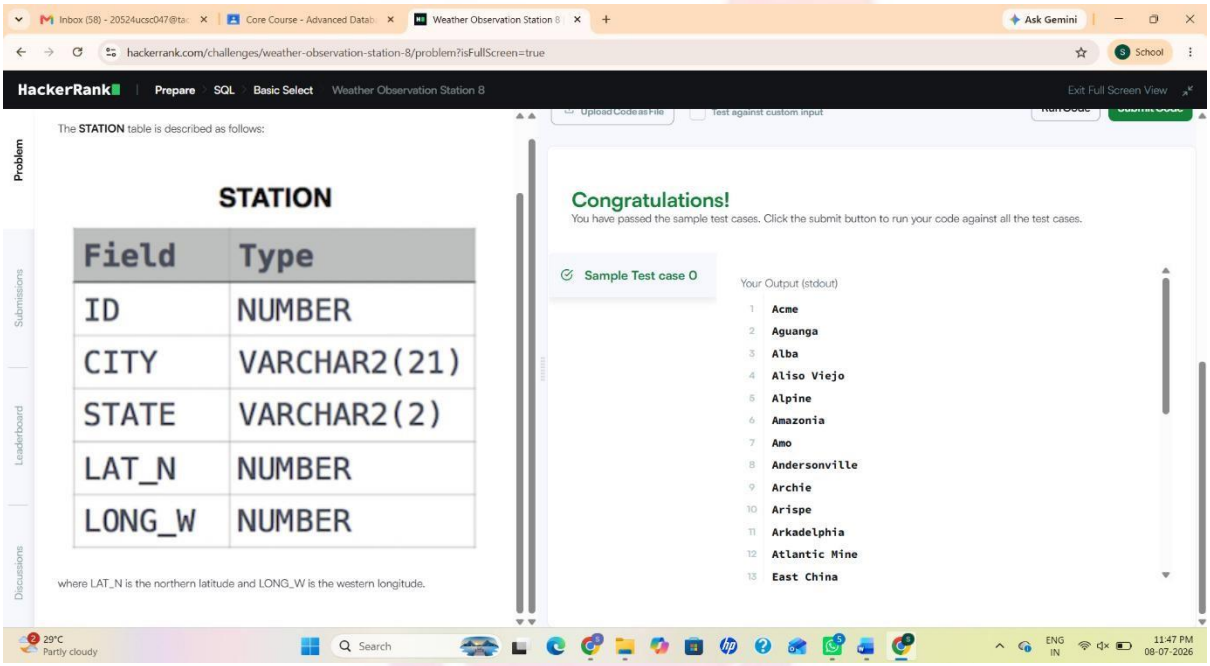


12. WEATHER OBSERVATION STATION

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) IN
('A','E','I','O','U') AND UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
```

OUTPUT

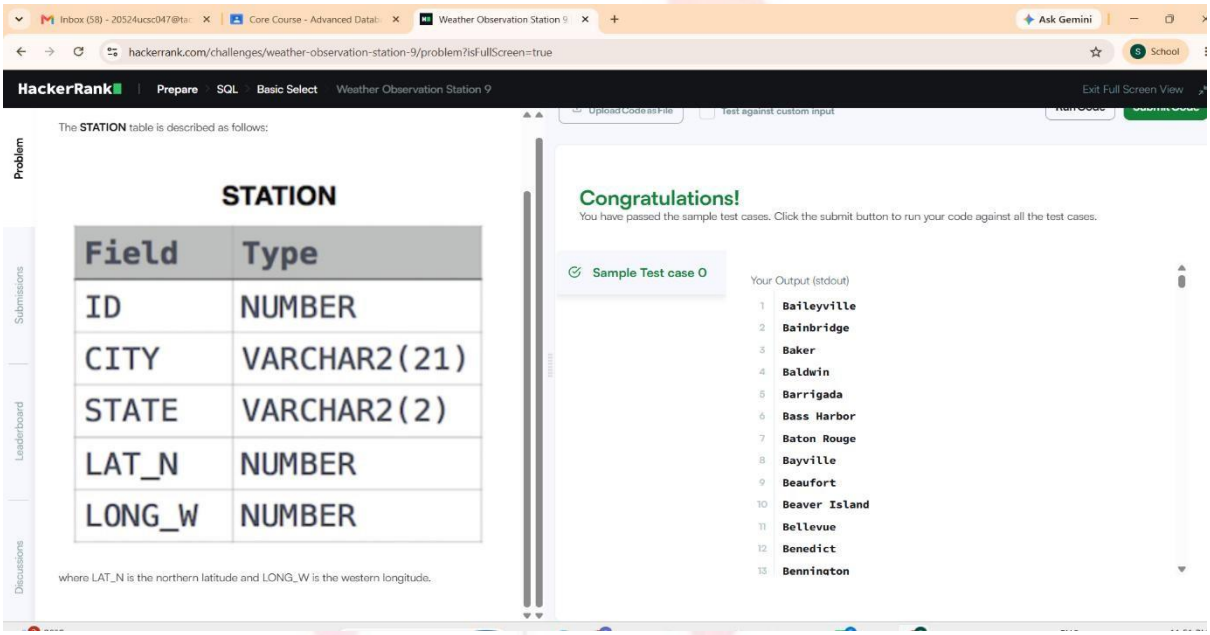


13. WEATHER OBSERVATION STATION

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT
IN ('A','E','I','O','U');
```

OUTPUT

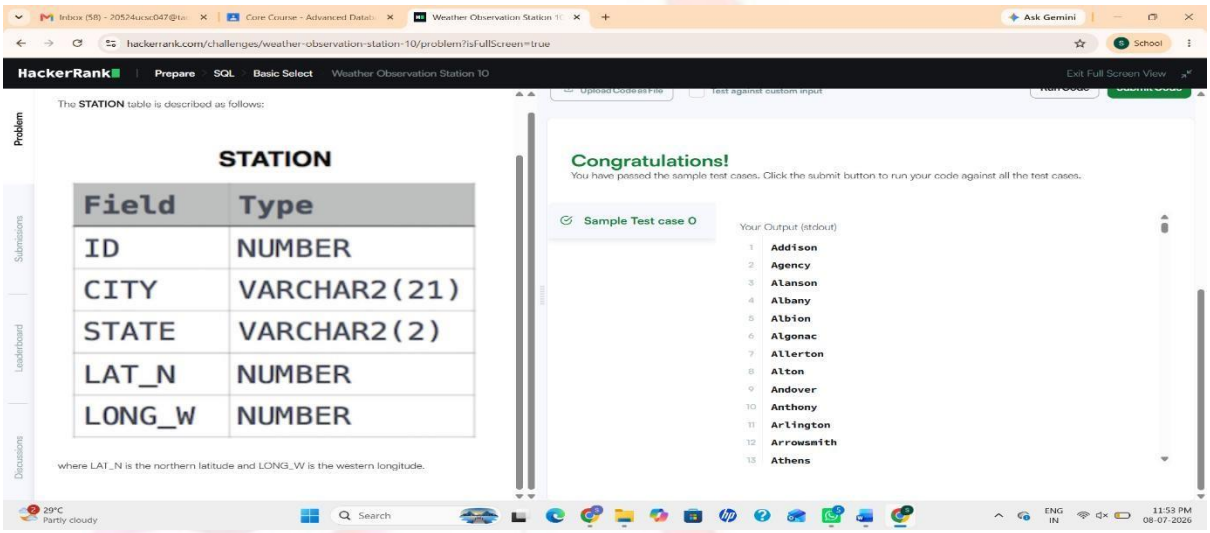


14. WEATHER OBSERVATION STATION

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN ('A','E','I','O','U');
```

OUTPUT

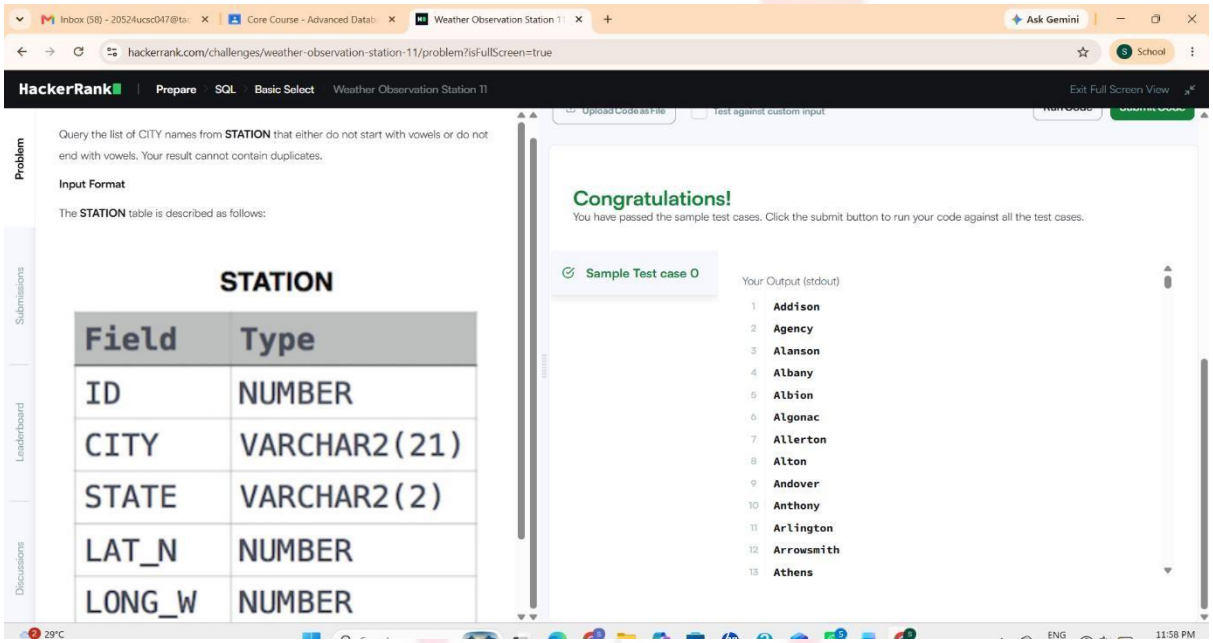


15.WEATHER OBSERVATION STATION 11

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT IN ('A','E','I','O','U') OR UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN ('A','E','I','O','U');
```

OUTPUT

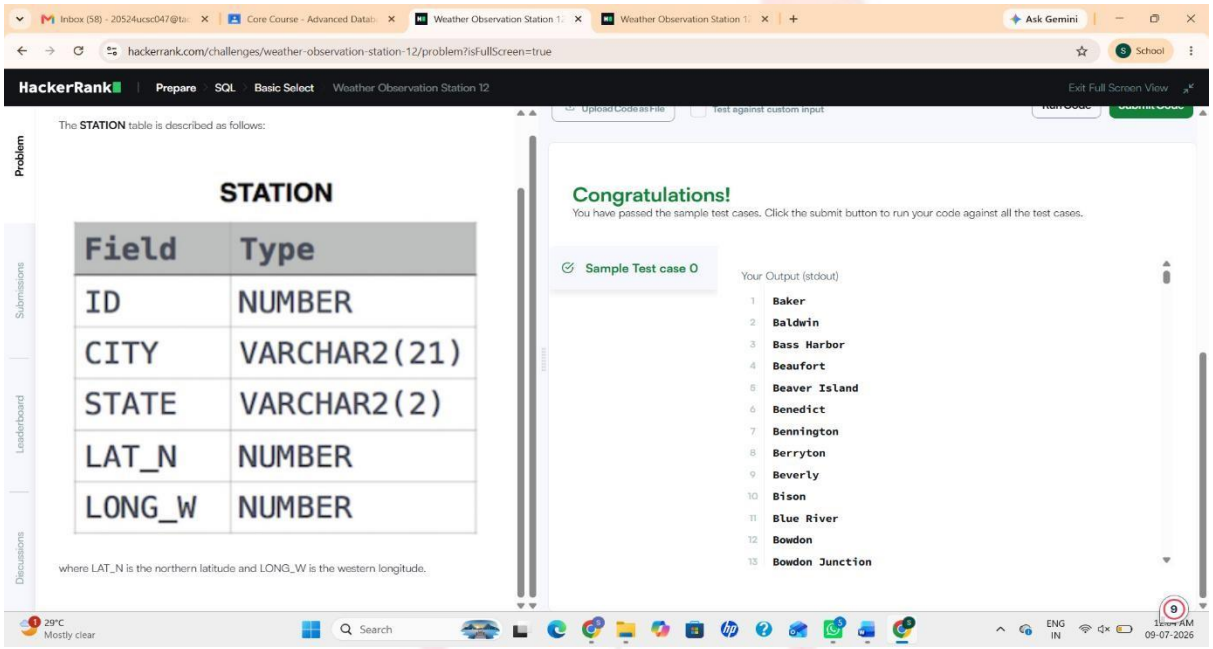


16.WEATHER OBSERVATION STATION 12

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT IN ('A','E','I','O','U') AND UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN ('A','E','I','O','U');
```

OUTPUT

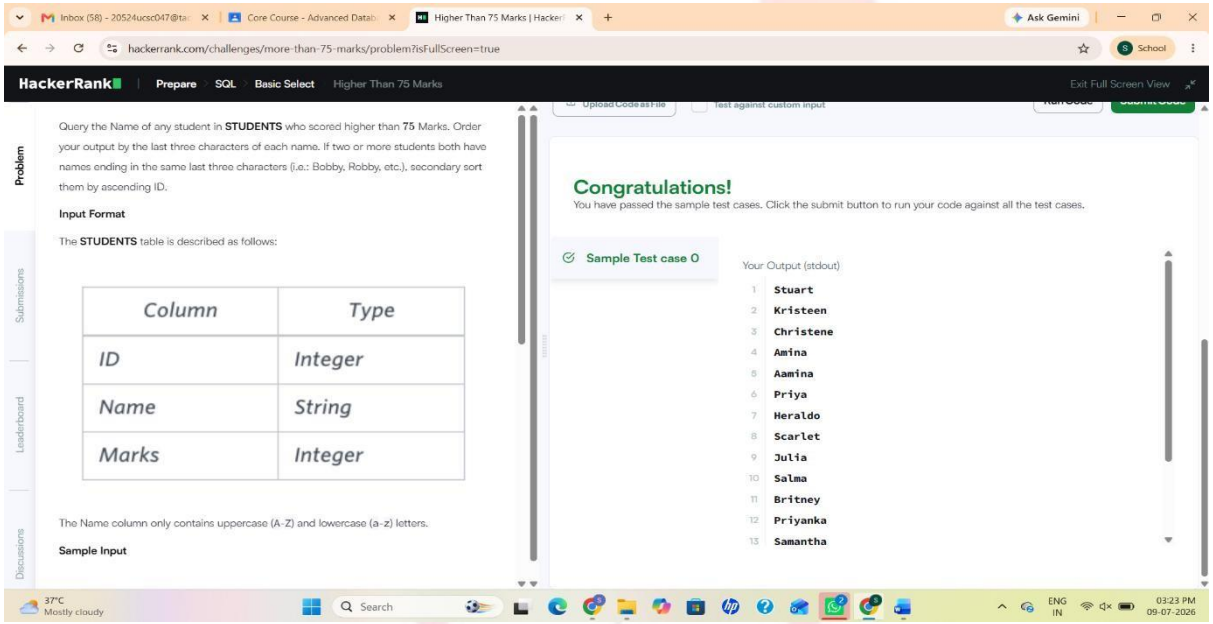


17. HIGHER THAN 75 MARKS

PROGRAM

```
SELECT Name FROM STUDENTS WHERE Marks > 75 ORDER BY SUBSTR(Name, LENGTH(Name)-2, 3), ID;
```

OUTPUT

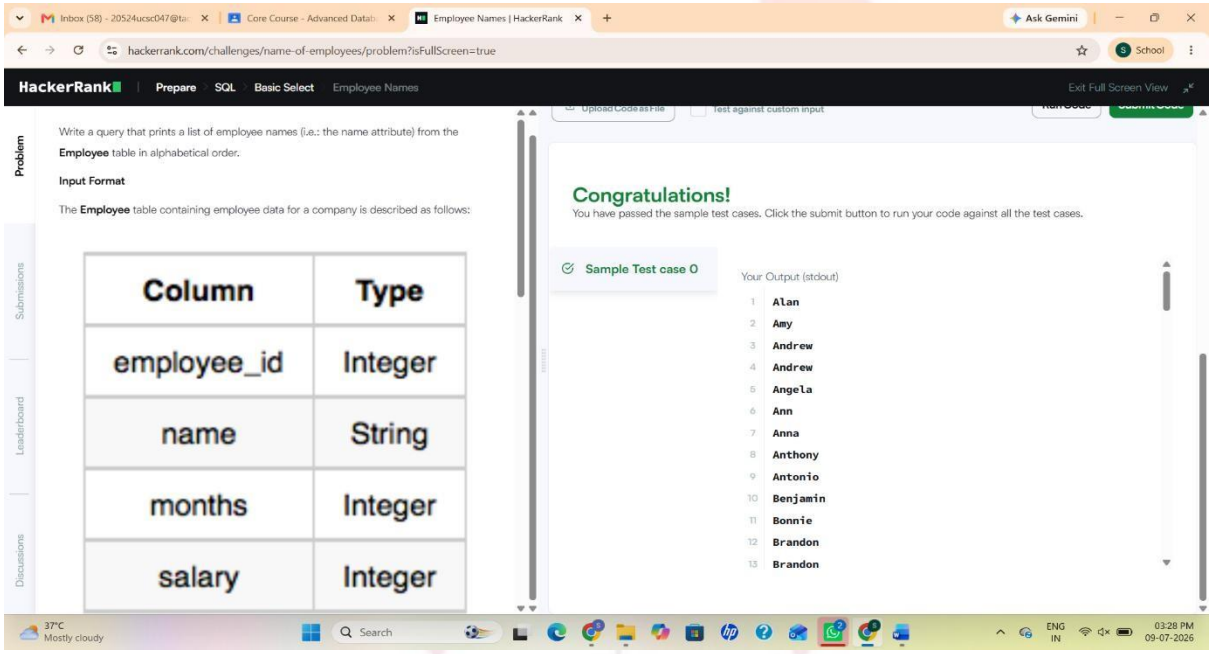


18.EMPLOYEE NAMES

PROGRAM

```
SELECT name FROM Employee ORDER BY name;
```

OUTPUT

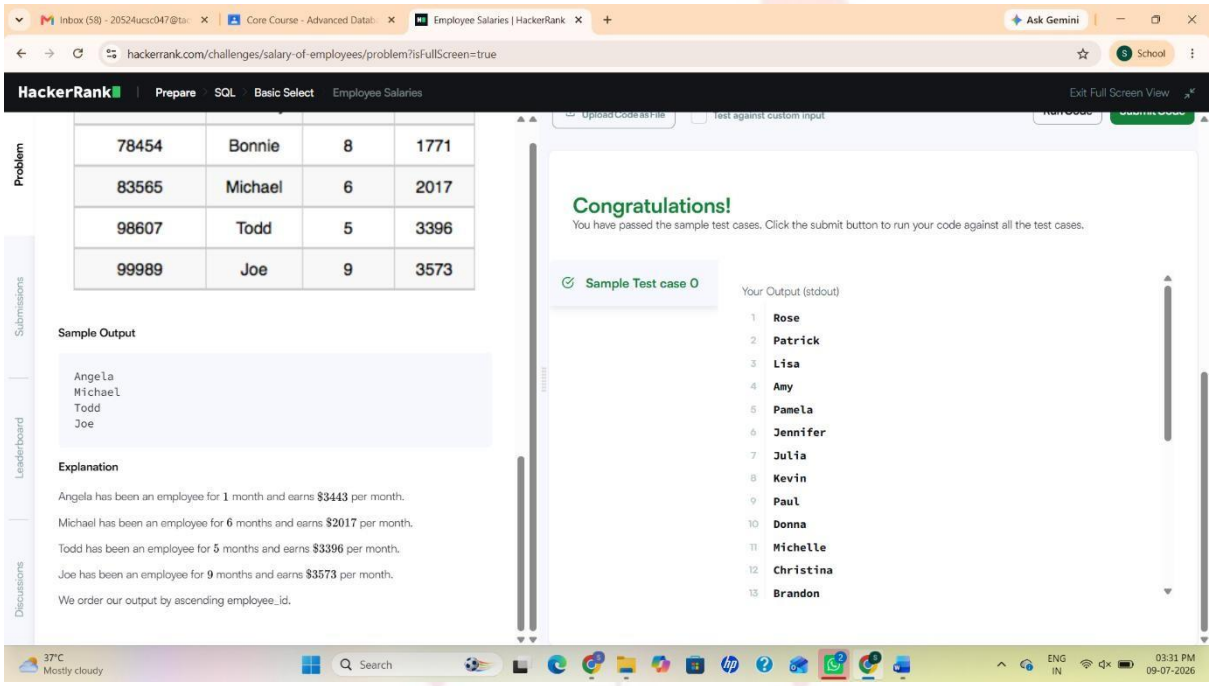


19.EMPLOYEE SALARIES

PROGRAM

```
SELECT name FROM Employee WHERE salary > 2000 AND months < 10 ORDER BY employee_id;
```

OUTPUT

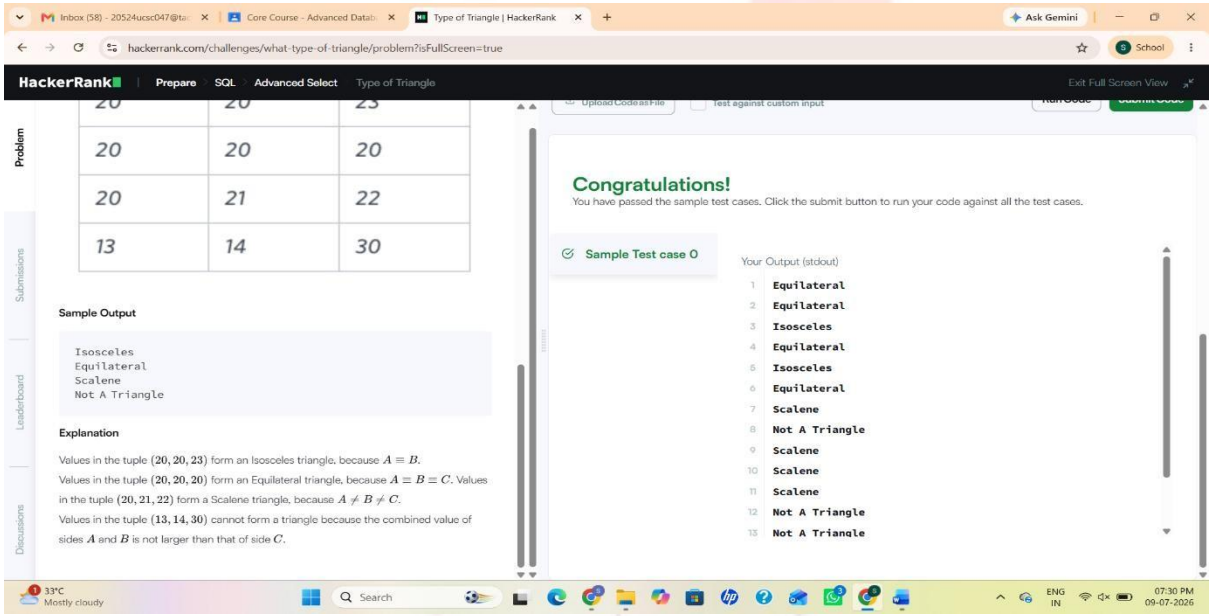


20.TYPES OF TRIANGLE

PROGRAM

```
SELECT CASE WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
WHEN A = B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN
'Isosceles' ELSE 'Scalene' END FROM TRIANGLES;
```

OUTPUT

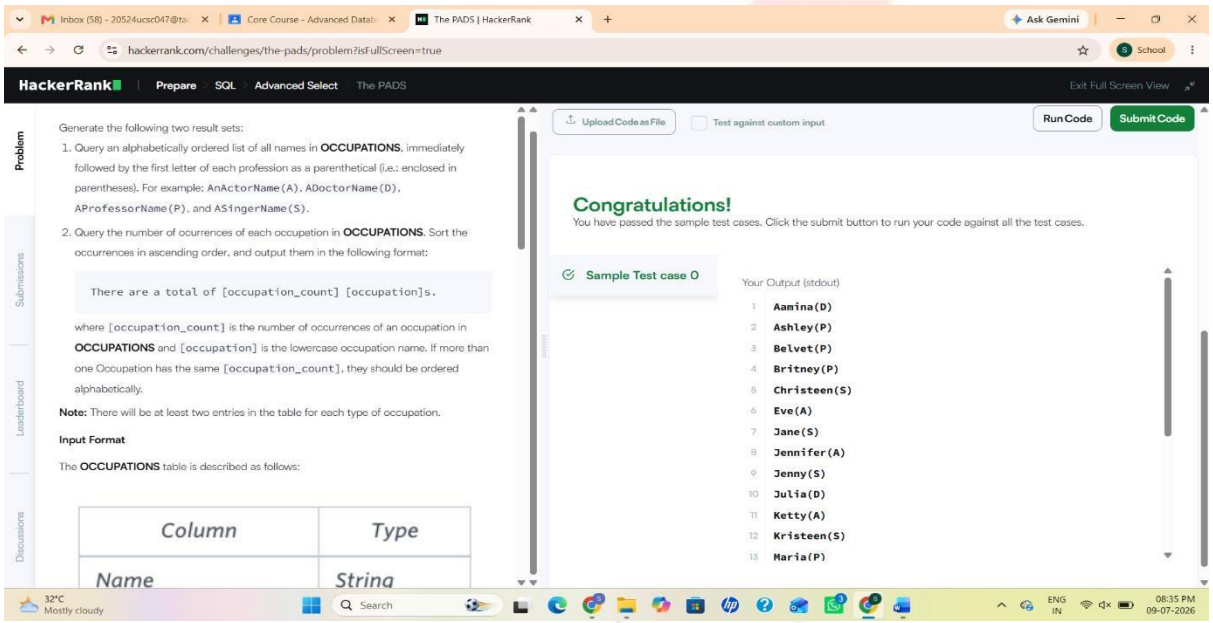


21.THE PADS

PROGRAM

```
SELECT NAME || '(' || SUBSTR(OCCUPATION,1,1) || ')' FROM OCCUPATIONS ORDER
BY NAME;
SELECT 'There are a total of ' || COUNT(*) || ' ' || LOWER(OCCUPATION) || 's.' FROM
OCCUPATIONS GROUP BY OCCUPATION ORDER BY COUNT(*), OCCUPATION;
```

OUTPUT

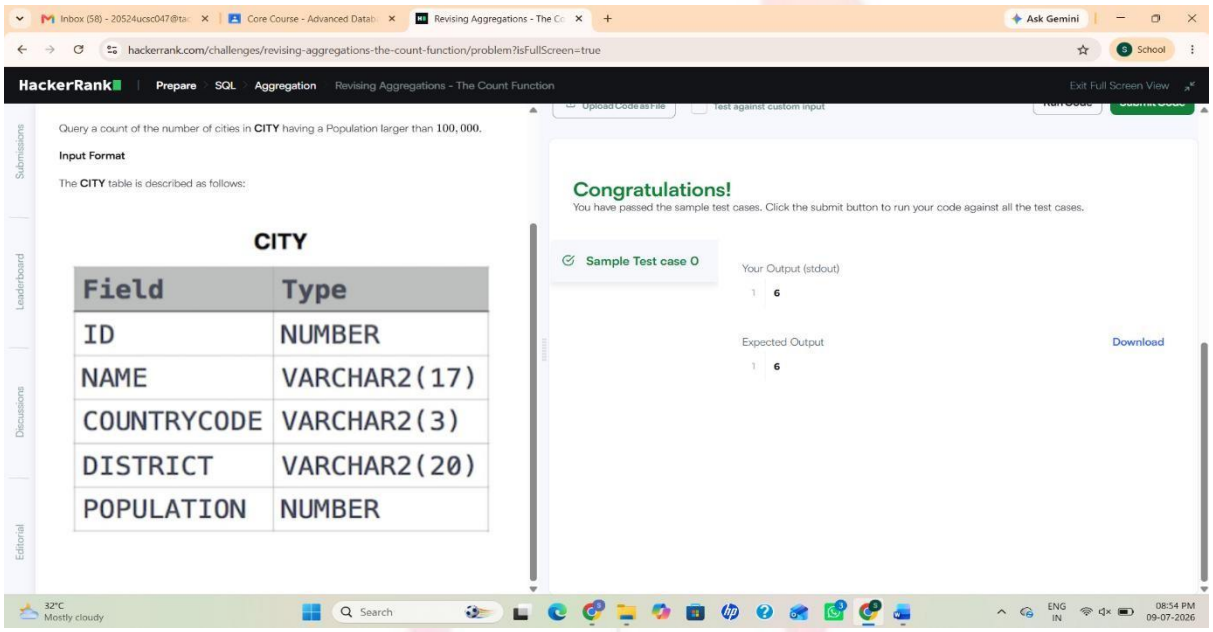


22.REVISING AGGREGATIONS – THE COUNT FUNCTION

PROGRAM

```
SELECT COUNT(*)
FROM CITY
WHERE POPULATION > 100000;
```

OUTPUT

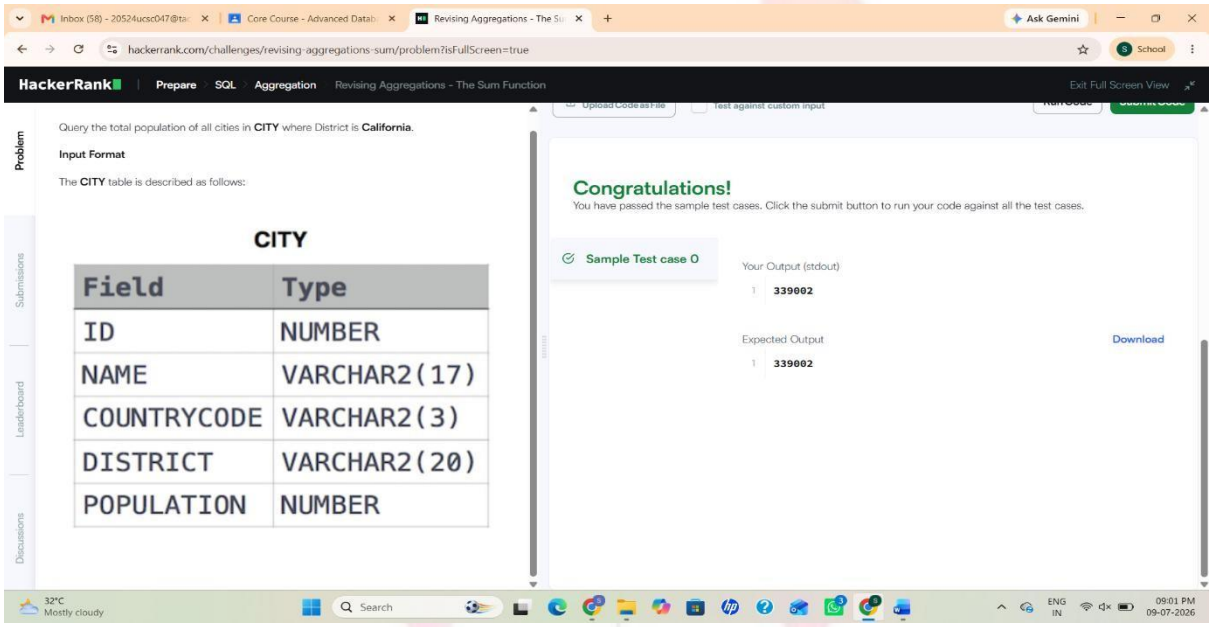


23. REVISING AGGREGATIONS – THE SUM FUNCTION

PROGRAM

```
SELECT SUM(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT

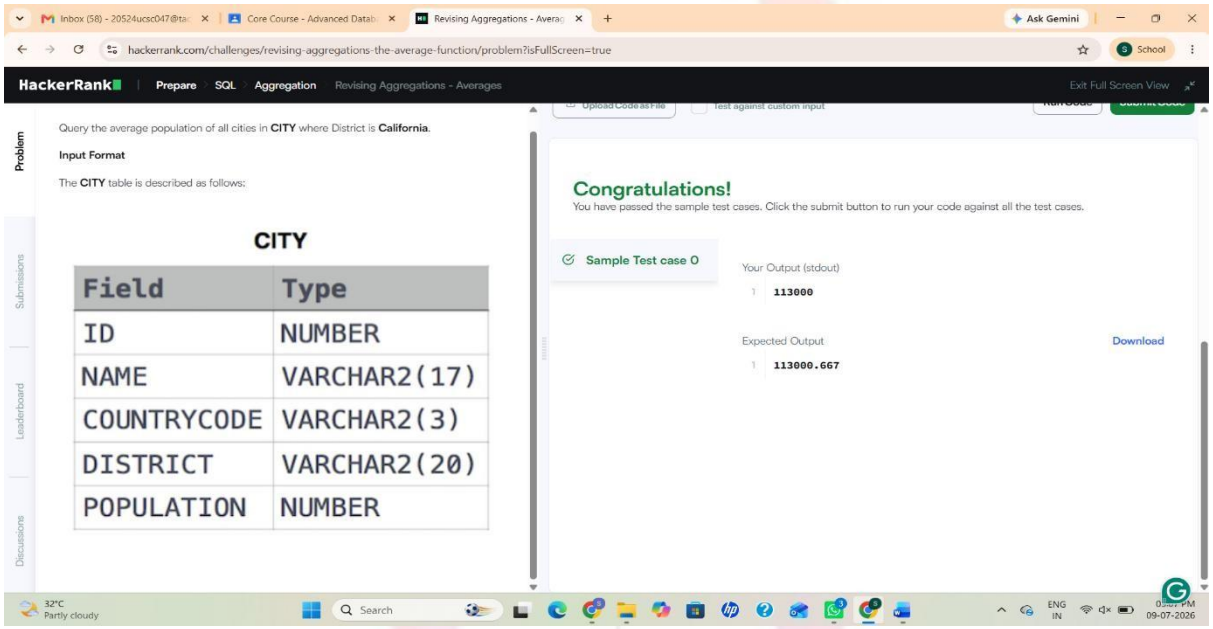


24. REVISING AGGREGATIONS – AVERAGES

Program

```
SELECT AVG(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT

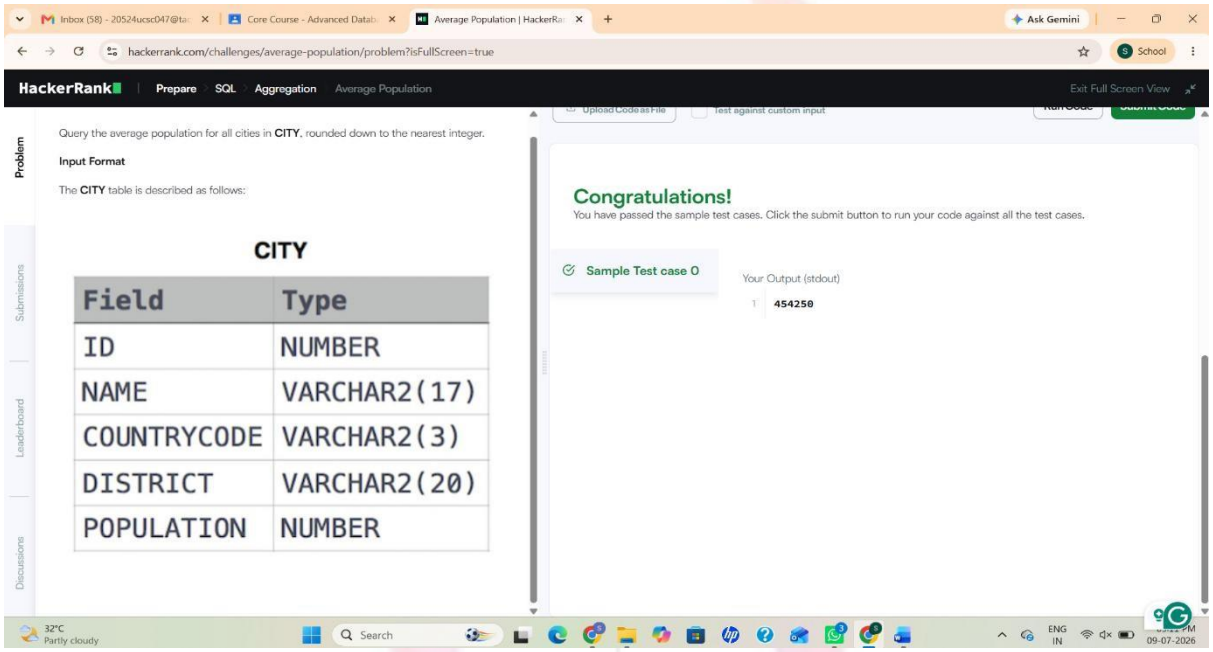


25.AVERGE POPULATION

PROGRAM

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT

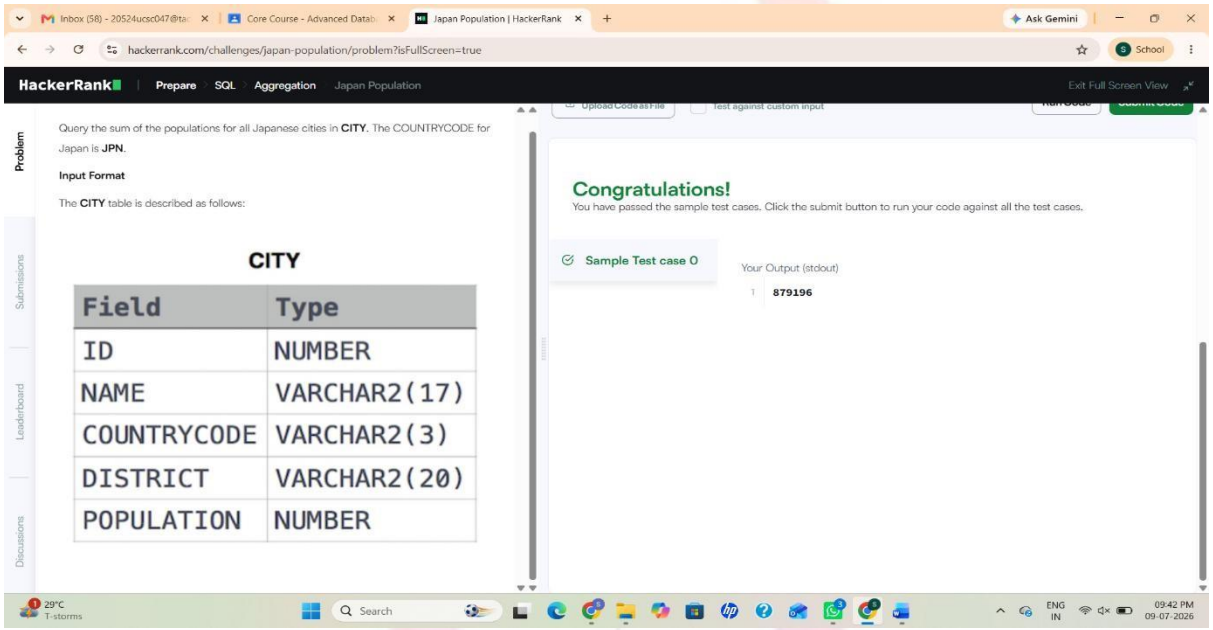


26.JAPAN POPULATION

program

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT

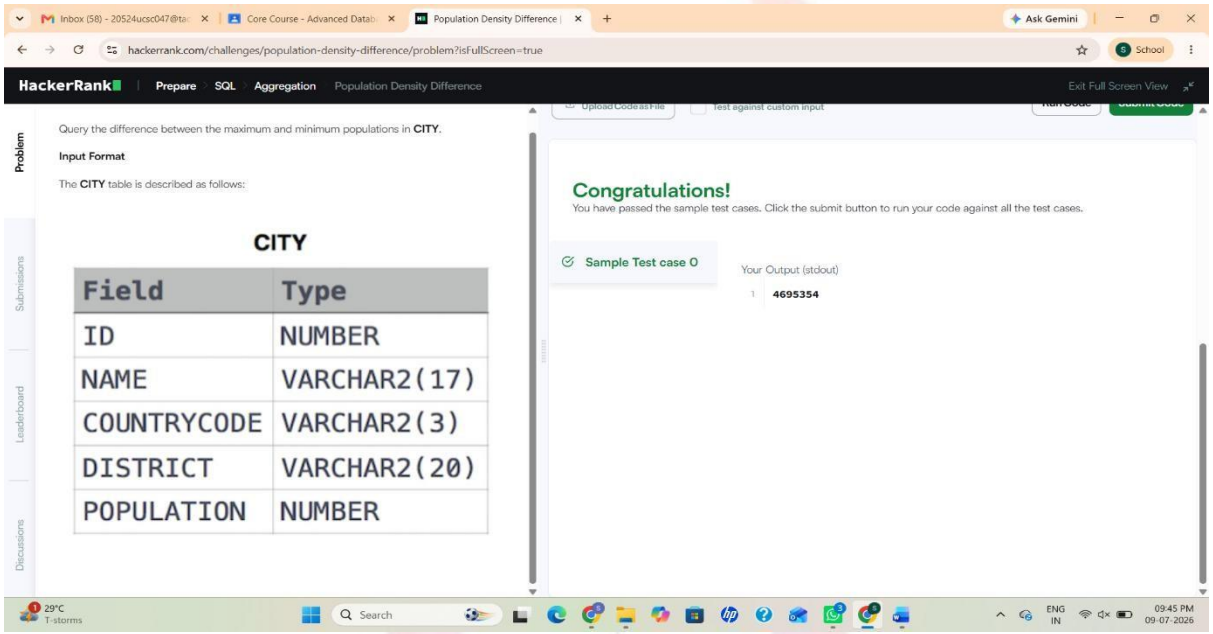


27.POPULATION DENSITY DIFFERENCE

PROGRAM

```
SELECT MAX(POPULATION) - MIN(POPULATION)
FROM CITY;
```

OUTPUT

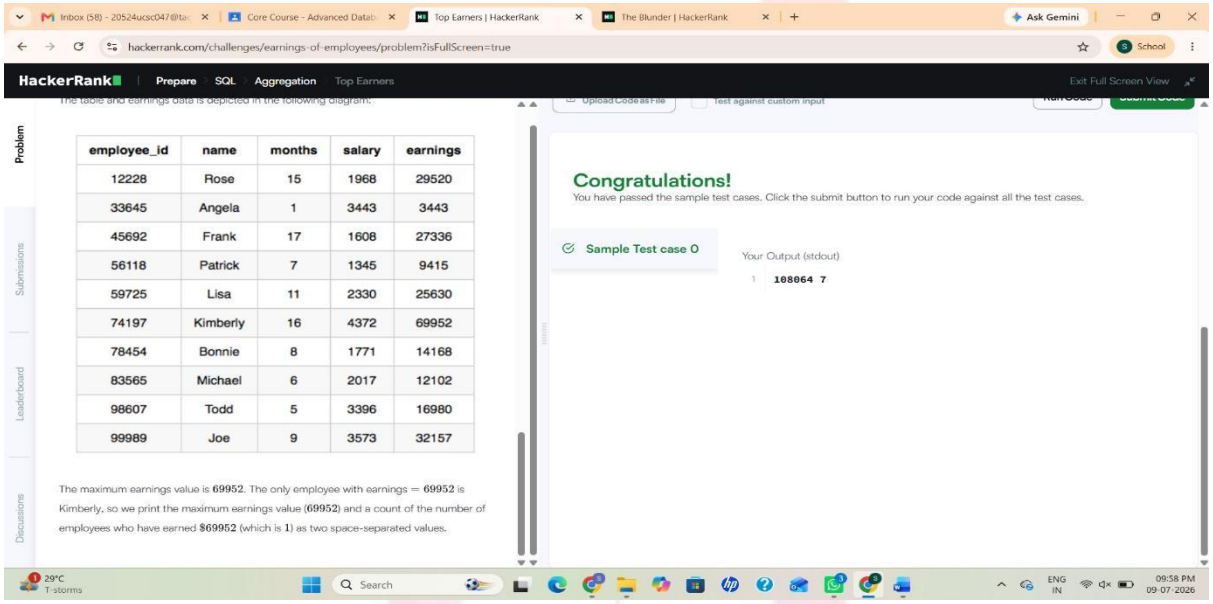


28.TOP EARNERS

Program

SELECT MAX(salary * months), COUNT(*) FROM Employee WHERE salary * months = (SELECT MAX(salary * months) FROM Employee);

OUTPUT

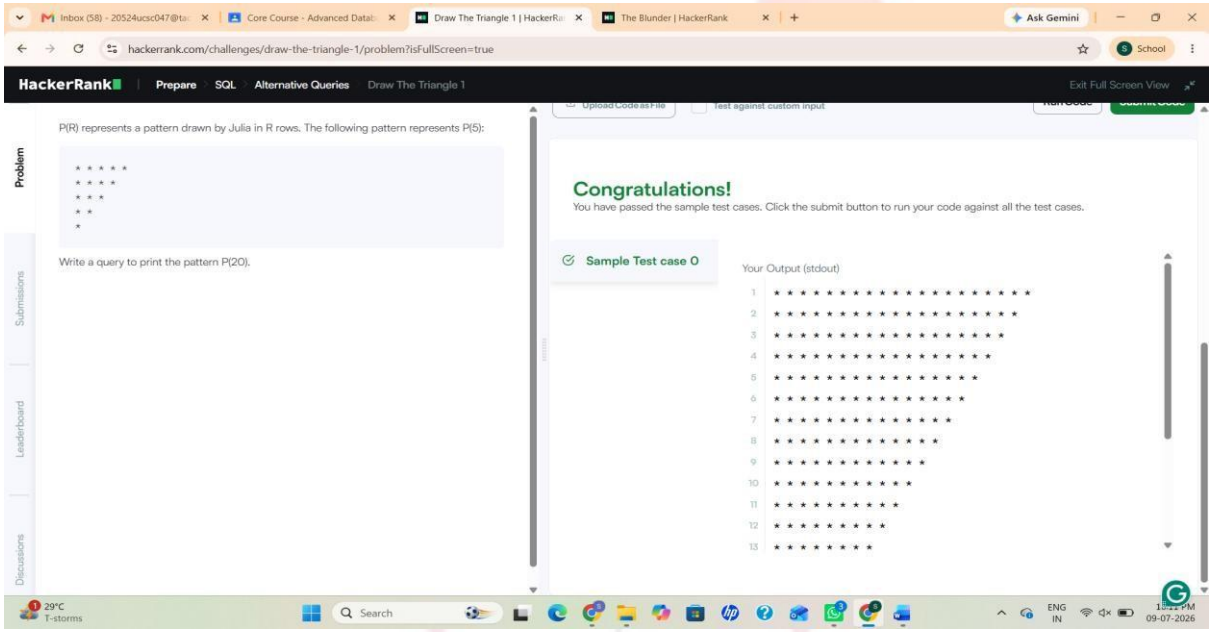


29.DRAW THE TRIANGLE 1

PROGRAM

```
SET @n = 21;
SELECT REPEAT('* ', @n := @n - 1)
FROM information_schema.tables
LIMIT 20;
```

OUTPUT

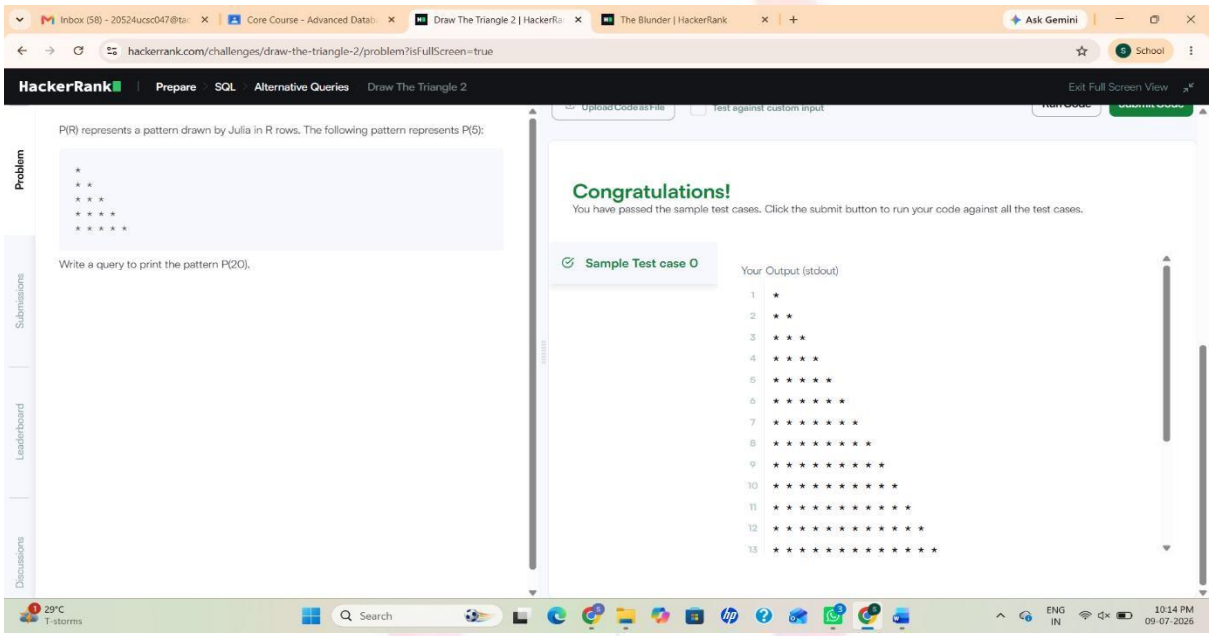


30. DRAW THE TRIANGLE 2

PROGRAM

```
SELECT REPEAT('* ', @n := @n + 1)
FROM information_schema.tables,
(SELECT @n := 0) t
LIMIT 20;
```

OUTPUT



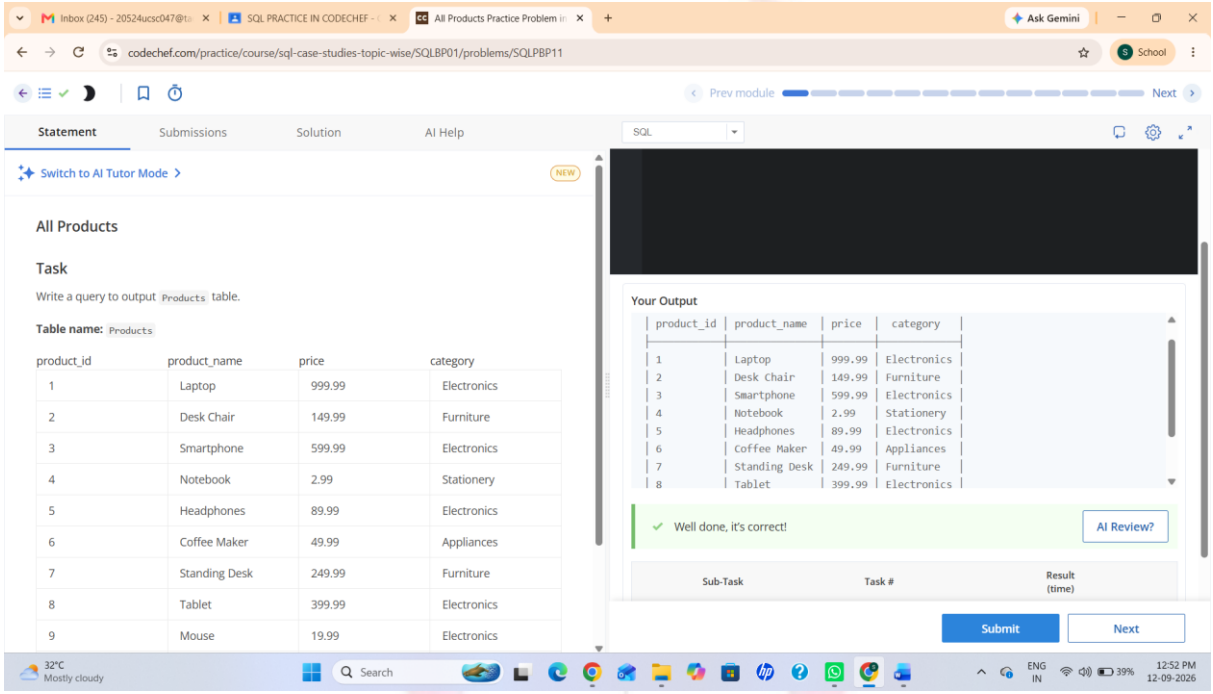
CODE CHEF

1.ALL PRODUCTS

QUERY

```
SELECT * FROM Products;
```

OUTPUT

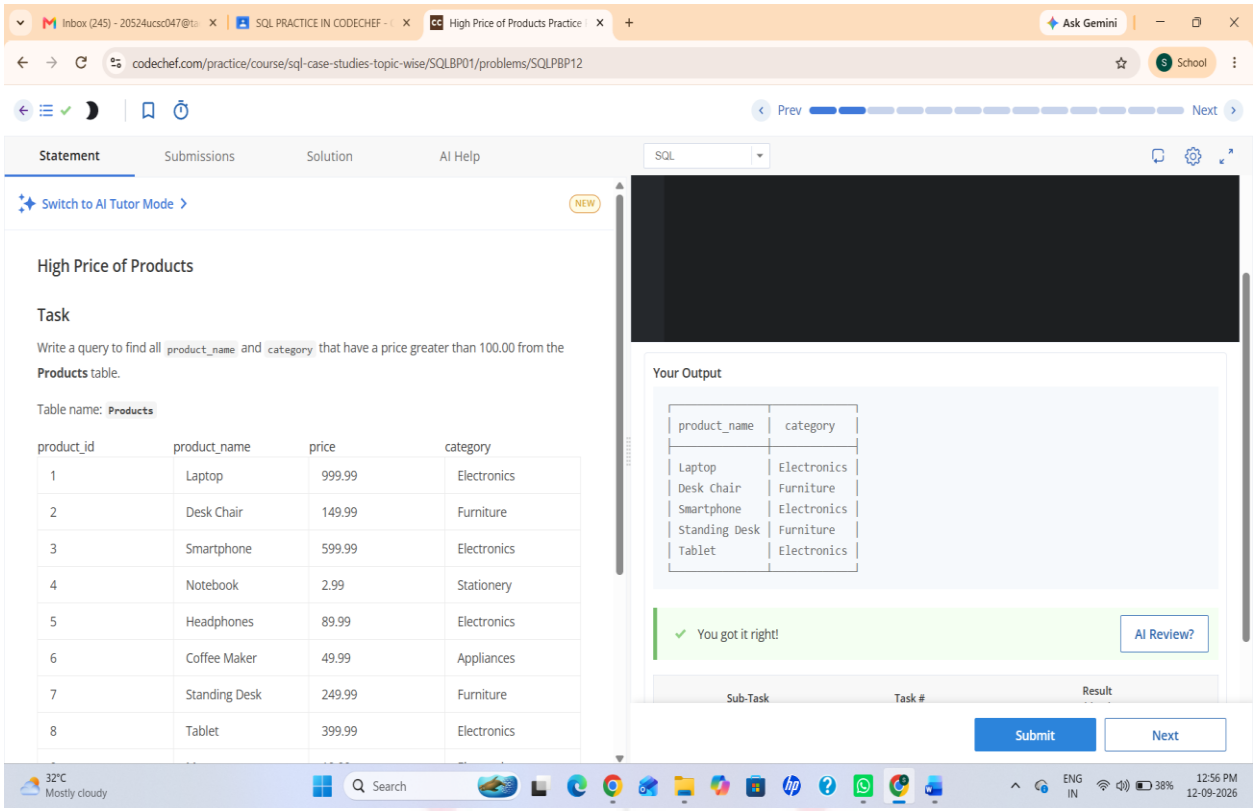


2.HIGH PRICE OF PRODUCTS

QUERY

```
SELECT product_name, category
FROM Products
WHERE price > 100.00;
```

OUTPUT

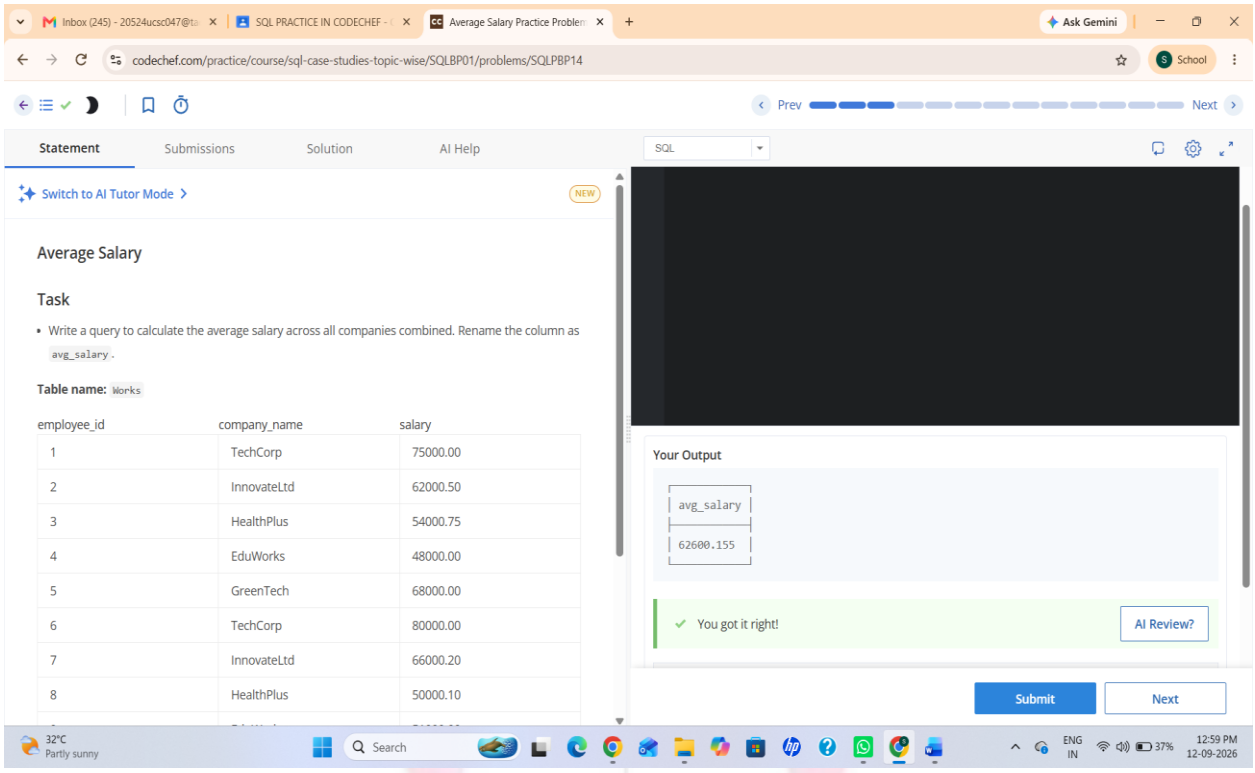


3.AVERAGE SALARY

QUERY

```
SELECT AVG(salary) AS avg_salary
FROM Works;
```

OUTPUT

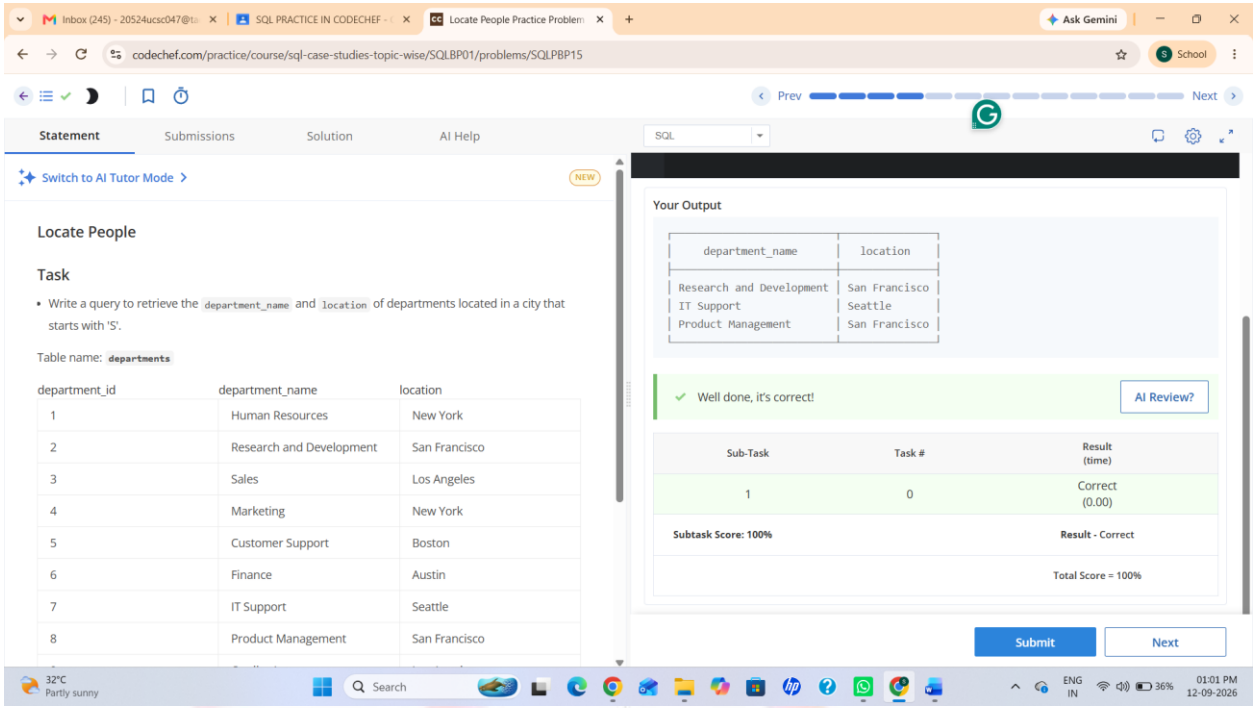


4.LOCATE PEOPLE

QUERY

```
SELECT department_name, location
FROM departments
WHERE location LIKE 'S%';
```

OUTPUT

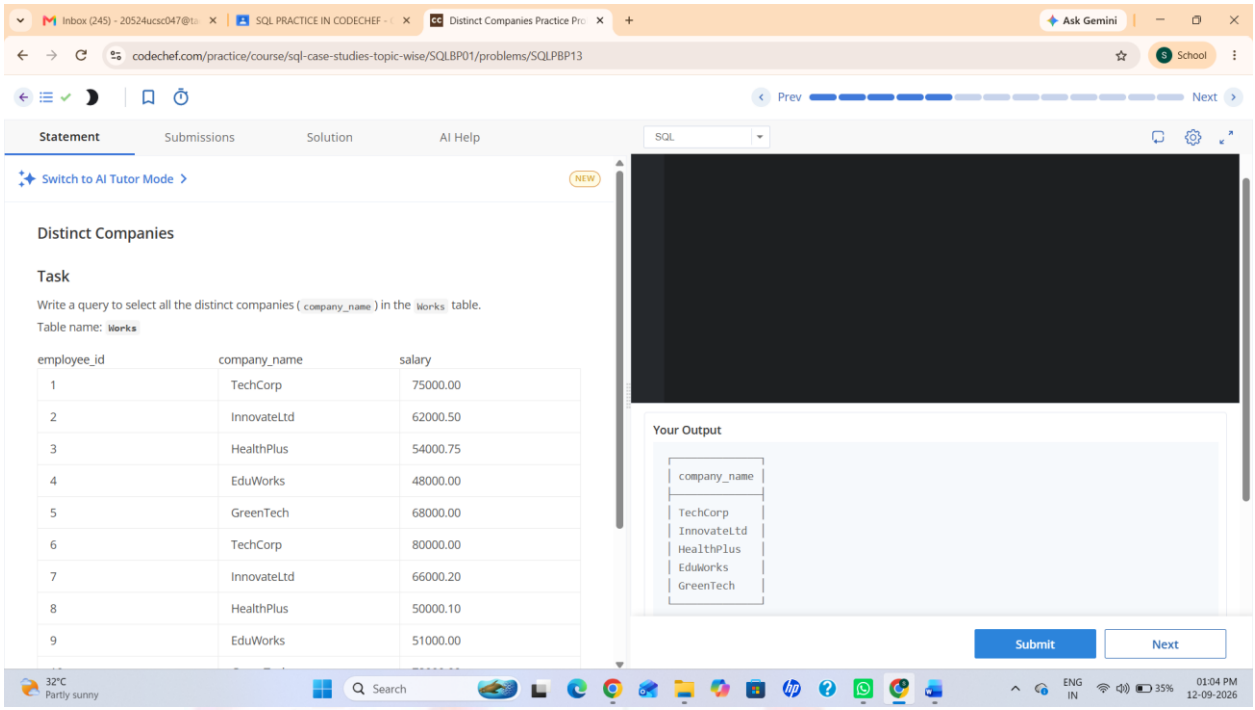


5. DISTINCT COMPANIES

QUERY

```
SELECT DISTINCT company_name
FROM Works;
```

OUTPUT

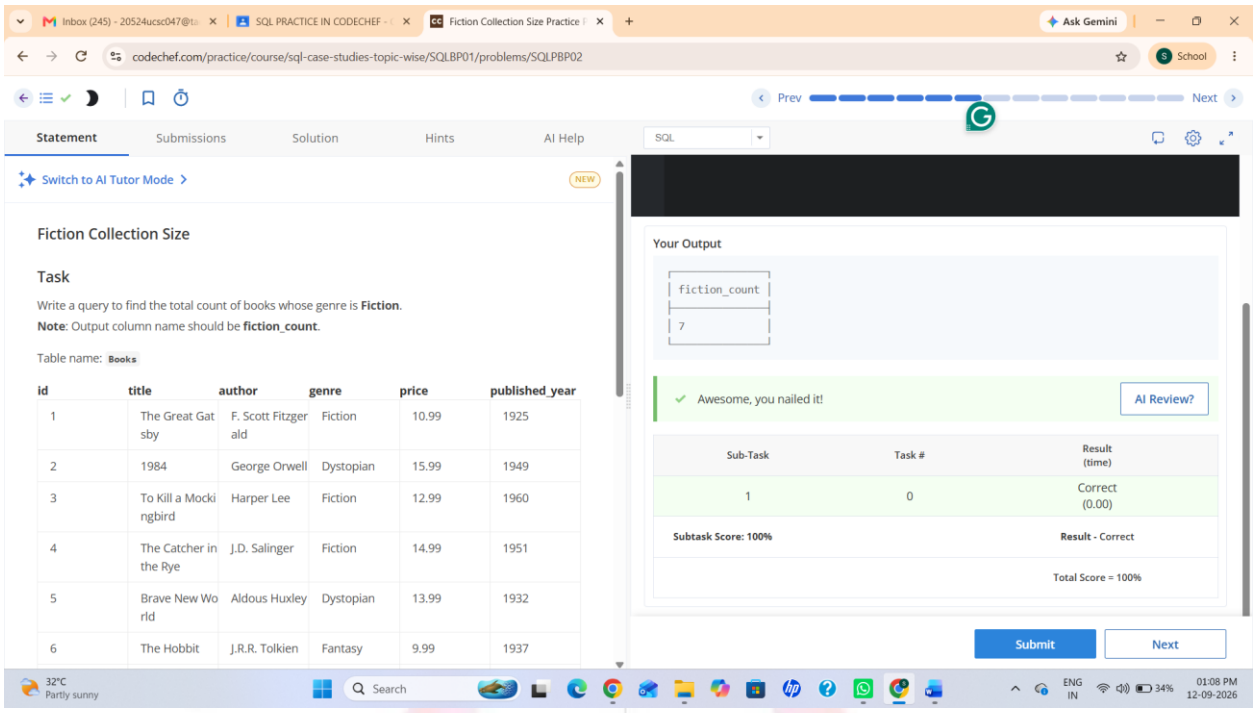


6.FICTION COLLECTION SIZE

QUERY

```
SELECT COUNT(*) AS fiction_count
FROM Books
WHERE genre = 'Fiction';
```

OUTPUT

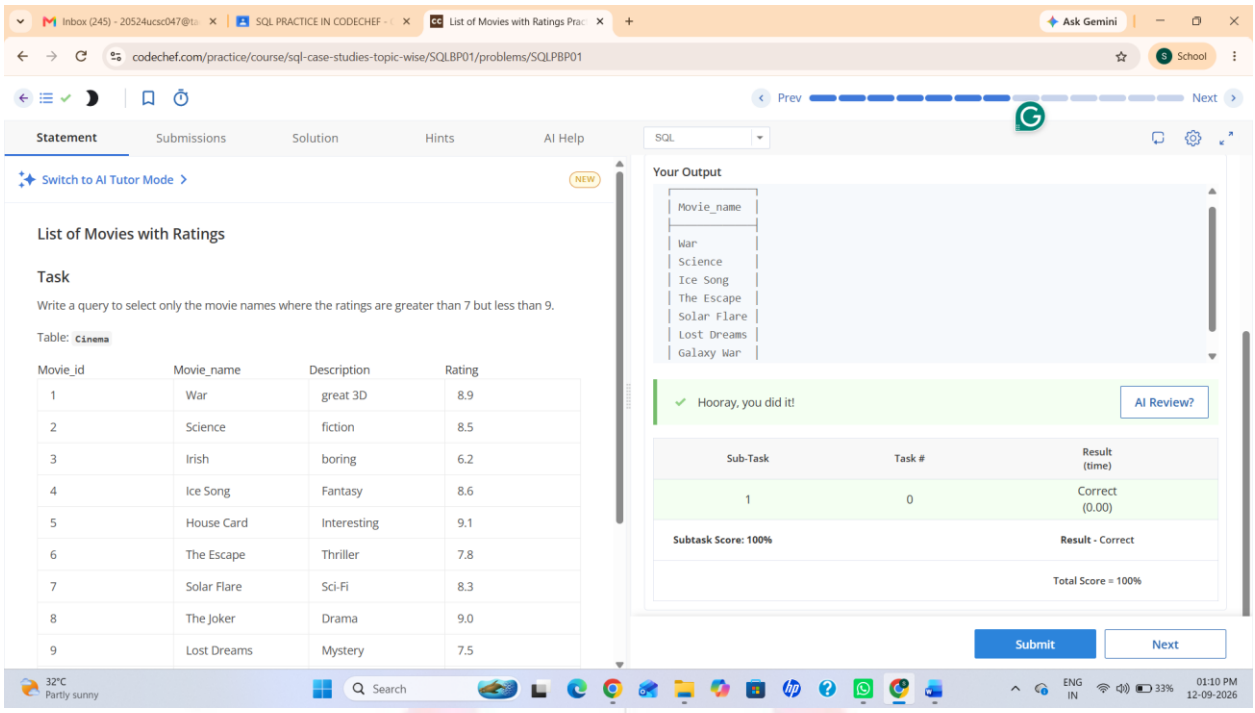


7.LIST OF MOVIES WITH RATINGS

QUERY

```
SELECT Movie_name
FROM Cinema
WHERE Rating > 7 AND Rating < 9;
```

OUTPUT

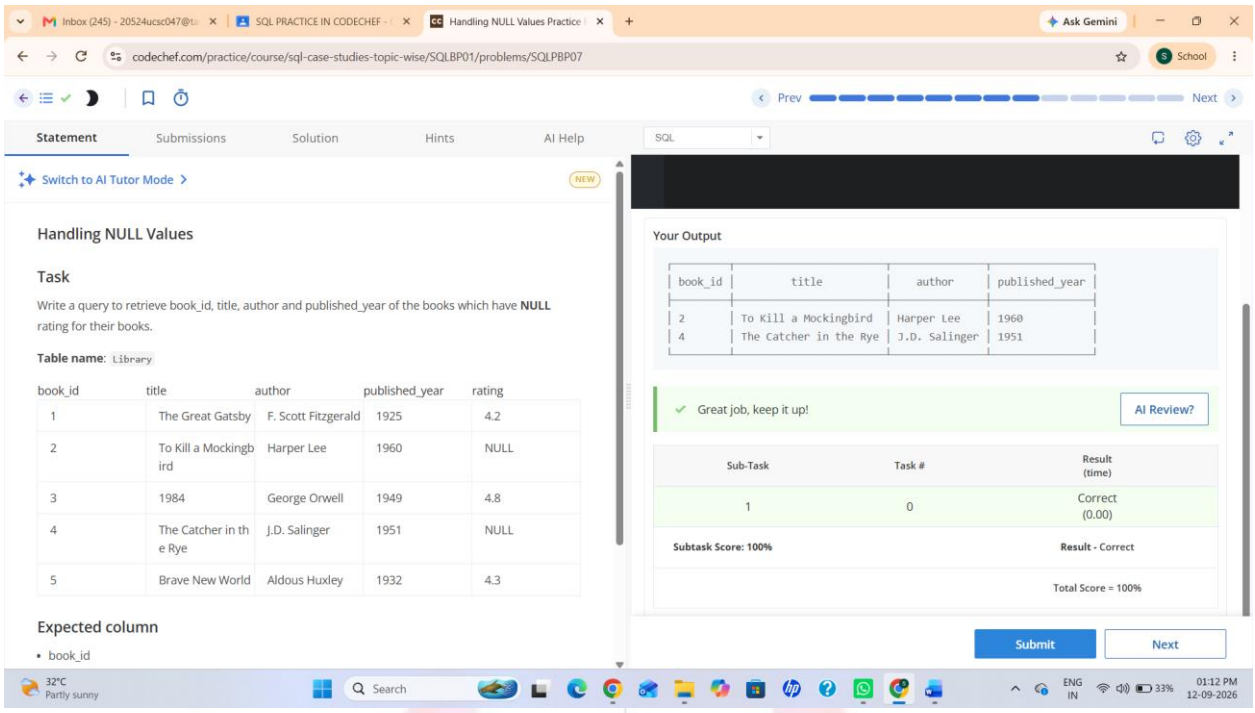


8.HANDLING NULL VALUES

QUERY

```
SELECT book_id, title, author, published_year
FROM Library
WHERE rating IS NULL;
```

OUTPUT

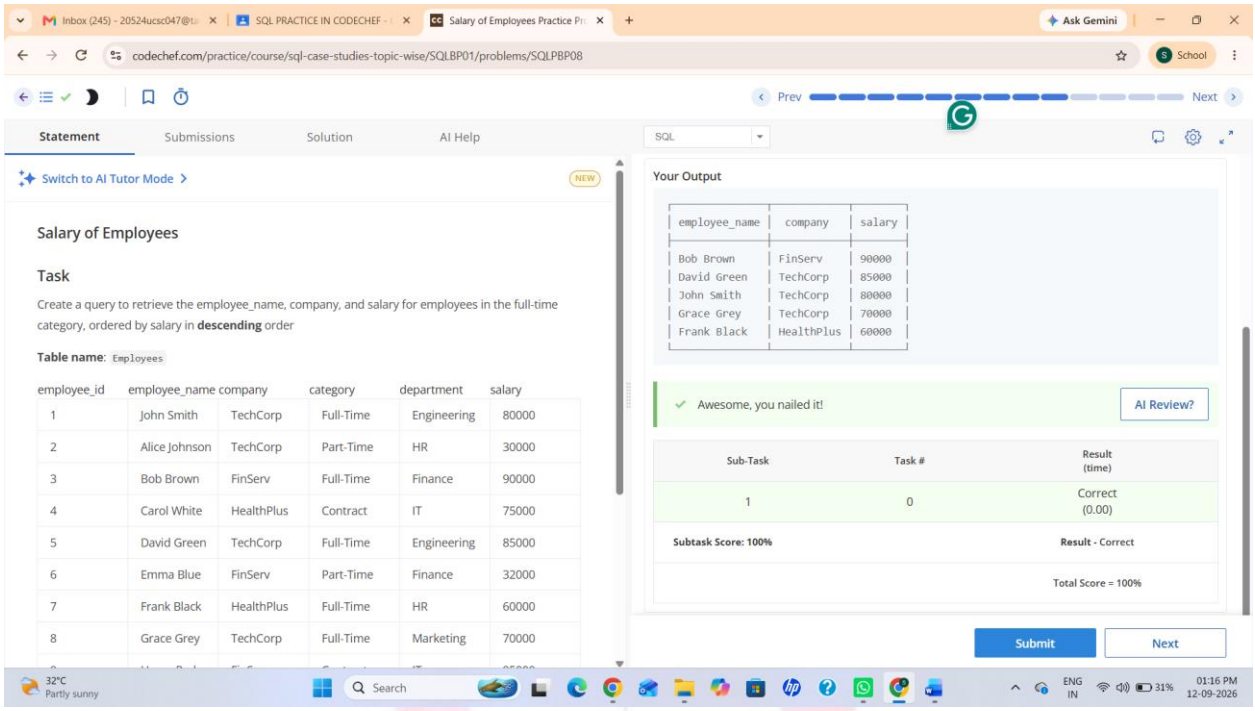


9.SALARY OF EMPLOYEES

QUERY

```
SELECT employee_name, company, salary
FROM Employees
WHERE category = 'Full-Time'
ORDER BY salary DESC;
```

OUTPUT

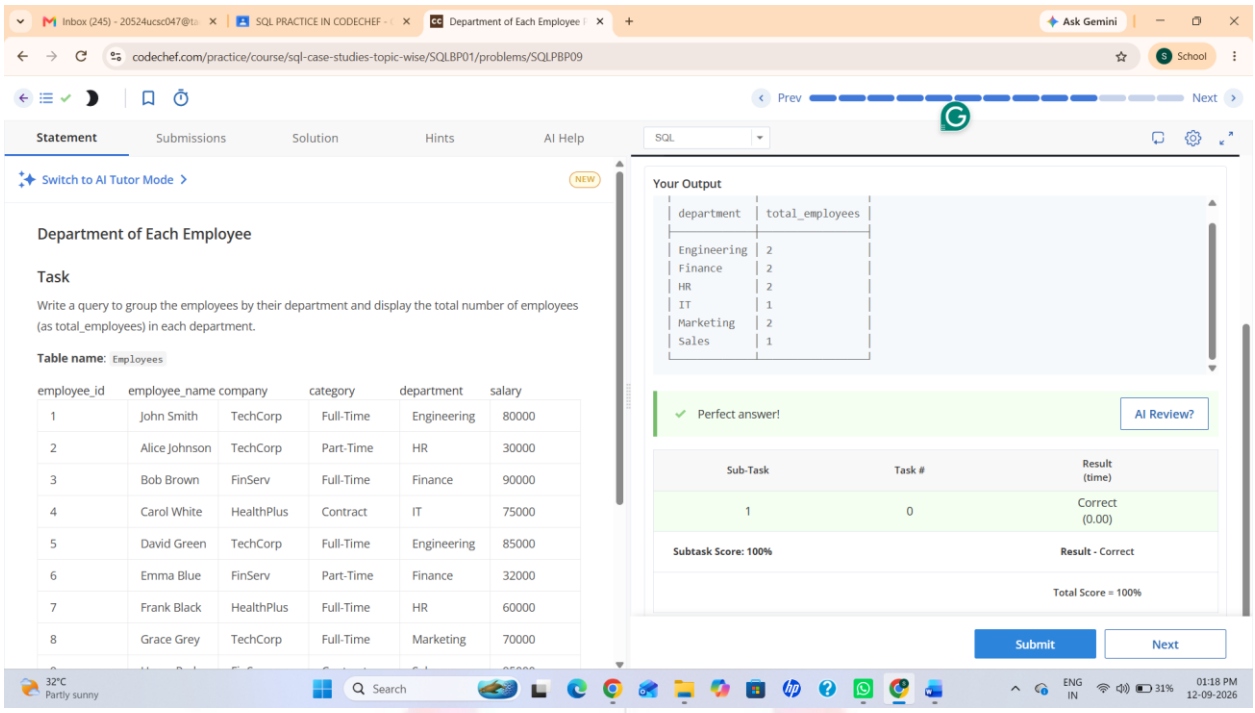


10. DEPARTMENT OF EACH EMPLOYEE

QUERY

```
SELECT department, COUNT(*) AS total_employees
FROM Employees
GROUP BY department;
```

OUTPUT

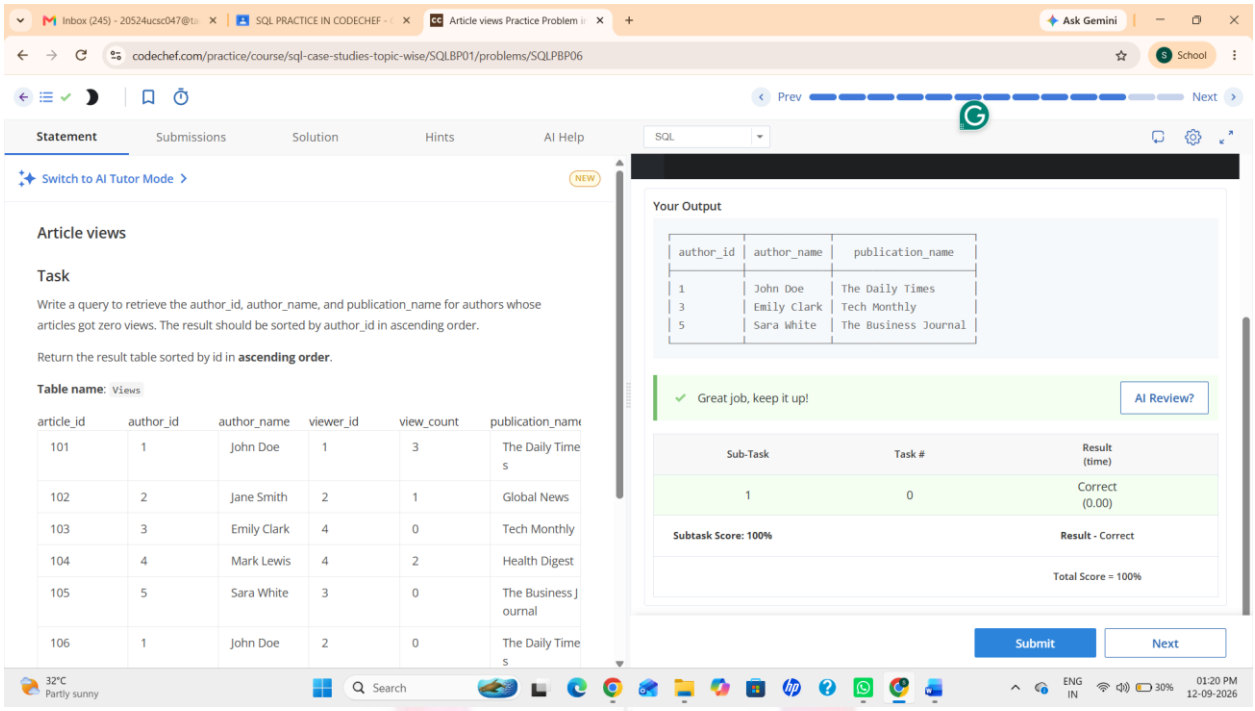


11.ARTICLE VIEWS

QUERY

```
SELECT author_id, author_name, publication_name
FROM Views
WHERE view_count = 0
ORDER BY author_id ASC;
```

OUTPUT

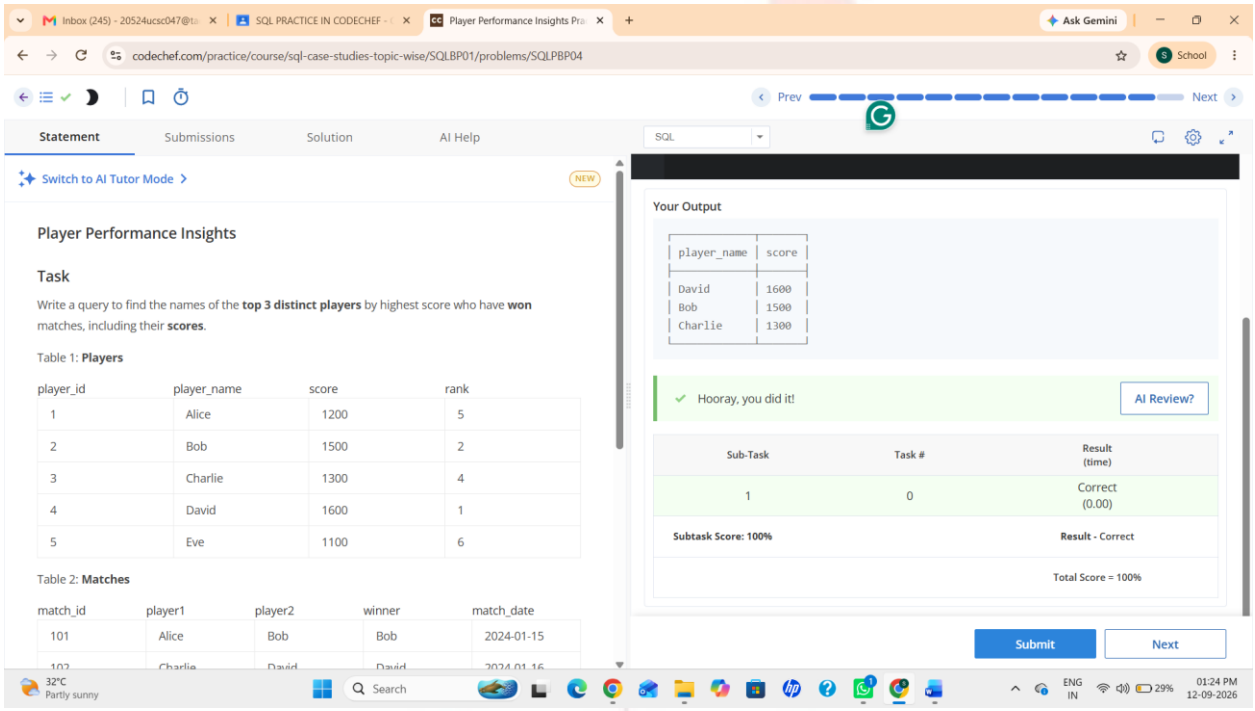


12.PLAYER PERFORMANCE INSIGHTS

QUERY

```
SELECT DISTINCT p.player_name, p.score
FROM Players p
JOIN Matches m
ON p.player_name = m.winner
ORDER BY p.score DESC
LIMIT 3;
```

OUTPUT



13.PLAYER DETAILS

QUERY

```
SELECT m.match_id,
       m.player_1,
       m.player_2,
       m.winner,
       m.match_date,
       p.score
FROM Matches m
JOIN Players p
ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT

